QuickSyn SCPI Controller and Telemetry Logger

William W. Wallace, NRAO/GBO

Version 0.0.10

Contents

1. Files **1**

2. Link parameters **4**
2.1 Transport: why pyserial, and what PyVISA would add 5

3. SCPI coverage **5**
3.1 Mandatory settling delays 7
3.2 Interlocks enforced 7
3.3 Output power control 7
3.4 RF OUTPUT OFF 9
3.5 10 MHz reference: source, output and LEDs 9
3.6 Typing values 10
3.7 Calibration (program-side) 11

4. The 64-byte buffer, and why frequencies are encoded two ways **15**

5. Logging **16**
5.1 Sample rate 16
5.2 Memory discipline 17
5.3 Parallelism and GPU 17
5.4 Live SCPI console 17
5.5 Auto-read of all settings 19

6. Veusz documents, reduction and plots **19**
6.1 Why `.vszh5` and not `.vsz` 20
6.2 The writer: `vszh5_writer.py` (from `isp_speed_monitor.py`) 20
6.3 What the session document contains 22
6.4 From the controller 22
6.5 Rendering, and Veusz command-line limitations 23
6.6 Statistics 24
6.7 PDF bookmarks and the LuaLaTeX session report 24

7. Model capability table **25**

8. Memory hygiene **26**

9. Documented conflicts and how they are handled **26**

10. Verification status **28**
10.1 First hardware session (FSL-0020, 2026-10-12) 33
10.2 Second hardware check (FSL-0020, 2026-10-13/14) 34

11. Revision history **35**

12. References **42**
Sourcing confidence 46

13. Open items **49**

14. Licence **50**
14.1 Third-party components 51

Version 0.0.10 · Python 3.12 · Qt.py → PySide6 · 2026-10-14

Author	William W. Wallace
Company	NRAO/GBO
Email	wwallace@nrao.edu
Licence	GPL-3.0-or-later — see `LICENSE` and §14
Versioning	Semantic Versioning 2.0.0 [15]; `0.y.z` means initial development
Status	Internal draft — hardware sessions **2026-10-12** and **2026-10-14** (FSL-0020, §10.1–§10.2); Calibration tab (§3.7) verified on the simulator only; FSW not yet tested

SCPI control of the NI / Phase Matrix QuickSyn® microwave frequency synthesiser family over the USB virtual COM port, with a timestamped log of every command generated and a telemetry stream sampled at a user-selected rate.

---

1. Files

File	Version	Purpose
`quicksyn_scpi_gui_v0_0_10.py`	0.0.10	Controller, logger and GUI with the live SCPI console (single file, Spyder `# %%` cells); output-power fix and RF OUTPUT OFF (§3.3, §3.4); hardware-session fixes, reference LEDs, auto-read, free typing (§3.5–§3.6, §5.5, §10.1); **Calibration tab** (§3.7) and the stale `STAT?` bit 5 of the FSL-0020 (§3.5)
`quicksyn_veusz_export_v0_0_10.py`	0.0.10	Reduction and statistics; `export_veusz()` writes the session `.vszh5`; renders PDF (with bookmarks) and PNG from it; writes and compiles the LuaLaTeX session report
`vszh5_writer.py`	0.0.10	**Veusz HDF5 writer** (`VeuszWriter`, `VszDoc`, `vz_text`, …) from `isp_speed_monitor.py`: h5py only, no Veusz needed to write; `render_pages()` and `add_pdf_bookmarks()`; reusable in other programs
`environment_quicksyn.yml`	0.0.10	Mamba environment definition
`README_quicksyn_controller.md`	0.0.10	This file

File	Version	Purpose
`README_quicksyn_controller.pdf` / `.tex`	0.0.10	This file typeset with LuaLaTeX (bookmark hierarchy of the sections, outline pane opens); the `.tex` is the editable LaTeX source
`LICENSE`	—	Full GNU GPL version 3 text, unmodified
`tests/check_release_metadata.py`	0.0.10	Headers, SemVer, SPDX, history entries, `LICENSE` hash, `--version`, About dialog, every file at one version
`tests/check_smoke.py`	0.0.10	Driver, simulator, logging, sampler and full-GUI checks, including the controller's own `.vszh5` export; output power (native and SCPI forms, read-back, fallback, model gating), Enter-to-apply, RF OFF, console Send line; 0.0.8 hardware-session fixes (typed entries, combo-box enums, reference read-backs and LEDs, auto-read, temperature flag, *IDN? order); 0.0.9 10 MHz output path (native command kept, unanswered `OUTP:ROSC:STAT?`)
`tests/check_reduction.py`	0.0.10	Rate, jitter, ADEV and moving-statistics maths
`tests/check_vszh5.py`	0.0.10	Writer and `export_veusz()`: layout, types, tags, escaping, DST, bit-exact data, pages; render plumbing (Windows paths, broken or unstartable Veusz, fallback); PDF page scale, bookmarks, re-render; LuaLaTeX report; real-Veusz load, re-save comparison and renders when Veusz is available
`tests/check_console.py`	0.0.10	The live SCPI console: line format and alignment, thread safety, a simulated session against `transactions.csv`, a 200 000-exchange pause (memory and Resume time), filter, clear, save
`tests/check_calibration.py`	0.0.10	Calibration (§3.7): path loss (constant, CSV, Touchstone v1/v2 DB/MA/RI), tables, interpolation, level correction, OCXO tuning, per-unit store, PyVISA reader with a fake meter and counter; the GUI on a simulated FSL-0020 and FSW-0020; the stale `STAT?` bit 5; NI-panel command vectors

File	Version	Purpose
`tests/run_all.py`	0.0.10	Runs the six checks in parallel and prints a summary
`upstream/veusz-4.2.1-vszh5-load-quit-pdfdpi.patch`	—	Patch for Veusz itself (not needed by this package): `.vszh5` in `--export` and `Load()`, `Quit()` in listen mode, PDF page scale (§6.5)
`upstream/VEUSZ_UPSTREAM_ISSUE.md`	—	Ready-to-file issue text for the Veusz project, with the test results
`upstream/check_veusz_patch.py`	0.0.10	Reproduces the three Veusz limits and checks the patch, for any set of Veusz programs
`tools/build_readme_pdf.py`	0.0.10	Rebuilds the README PDF and `.tex` (pandoc + LuaLaTeX)
`examples/simulated_session_20261014_061313/`	0.0.10	Output of a 41 s **simulated** session recorded with 0.0.10 (four commanded frequency and power steps, each read back, RF OFF at the end, 10 Hz): `quicksyn_session.vszh5`, the bookmarked `quicksyn_session.pdf`, the overview PNG, `quicksyn_session_report.tex` and `.pdf` (LuaLaTeX), `reduction_summary.md`. No instrument was connected
`examples/hardware_session_20261012_100904_FSL-0020/`	0.0.9	The **first hardware session** (FSL-0020, 600 s at 2 Hz, recorded with 0.0.7) re-exported with the 0.0.9 export script: plots PDF, overview PNG, LuaLaTeX report, `reduction_summary.md` (§10.1). Shows the temperature artefact and the 15.6 ms latency quantisation
`examples/calibration_simulated_FSL-0020/`	0.0.10	A **simulated** power calibration (synthetic level $12 - 0.4\,f$/GHz dBm, 1.5 dB constant path loss, 20 points read by a fake meter): the table as JSON and CSV, its `.vszh5`, PDF and PNG, and screenshots of the Calibration tab and the Main tab's expected level (§3.7). **Not a measurement**

0.0.10 adds a **Calibration** tab (§3.7) in place of NI's GPIB-only calibration panel: power against frequency from typed readings or any SCPI power meter through PyVISA, with cable-loss correction from a constant, a CSV table or a Touchstone `.s2p`; and OCXO tuning with the documented `DIAG:CAL:REF:DAC` against a counter. The results are kept in this program per unit; nothing undocumented is written to the synthesizer. It also stops the false warning on the FSL-0020's 10 MHz output, whose `STAT?` bit 5 stays set while REF OUT switches (§3.5, §10.2).

0.0.9 corrected the 10 MHz output switch for the FSL-0020: its REF OUT does switch (confirmed with NI's panel and a power meter), but not with the SCPI command, so the native command is used once it has been seen to work, and a unit that does not answer `OUTP:ROSC:STAT?` no longer breaks the switch or *Read all settings* (§3.5).

0.0.8 fixes what the first hardware session found (§10.1): the reference source crash, frequencies below 1 GHz that could not be typed, the *IDN? field order, the 15.6 ms timing on Windows with Python 3.12, and an unchecked 10 MHz output switch. It adds Internal/External reference LEDs (§3.5) and an auto-read of all settings (§5.5). 0.0.7 fixed the output-level entry (§3.3) and added RF OUTPUT OFF (§3.4).

```
mamba env create -f environment_quicksyn.yml
mamba activate quicksyn
python quicksyn_scpi_gui_v0_0_10.py            # Help > About shows author, company, email, licence
python quicksyn_scpi_gui_v0_0_10.py --version  # prints the same identity block, no display needed
python tests/run_all.py                       # no instrument or display required
```

To include the checks that need a real Veusz program, point `QUICKSYN_TEST_VEUSZ_EXE` at it first (for example `veusz.exe`). The LuaLaTeX checks run when `lualatex` is found (§6.7).

Tick **Simulator mode** on the Connection tab to exercise the whole application, logging included, with no hardware attached.

---

2. Link parameters

Fixed by the vendor and not configurable on USB [1].

Parameter	Value
Baud rate	115200
Data bits / parity / stop bits	8 / none / 1
Flow control	none
Terminator	CR, decimal 13, `0x0D` — every command, every interface
Device buffer	**64 bytes including the terminator**
SCPI interfaces	USB, Ethernet, GPIB, RS232 (SPI is native-only)
Minimum firmware for SCPI	100

`DIAG:BAUD` retargets the **debug port** used by RS232, GPIB and Ethernet. It does *not* change the USB virtual COM port, so leave the baud selector at 115200 for a USB connection.

2.1 Transport: why pyserial, and what PyVISA would add

Once the NI driver is installed, the USB port "is utilized as a standard COM port (serial port) on the host PC" [1]. Talking to a COM port needs only a serial library, so the controller uses pyserial [32] directly. PyVISA [30] is a front end to a VISA library; with its pure-Python back end, pyvisa-py [31], a serial resource (`ASRL3::INSTR`) is itself implemented on top of pyserial. On USB, PyVISA would therefore add a layer over the same pyserial calls, not a capability. VISA's other advantages — IEEE 488.2 status reporting, `*OPC?`, service requests, an error queue — need instrument support the QuickSyn does not have (§3: only `*IDN?`, `*RST`, `*SAV`, `*RCL`), so none of them would apply.

Driving the port directly also keeps the details this instrument needs in one place: the 64-byte pre-flight (§4), the input-buffer flush before each command, the minimum gap between commands, a reader that accepts CR, LF or CRLF (the reply terminator is undocumented), and port listing with descriptions for the *Rescan ports* button (`serial.tools.list_ports`).

Where PyVISA *would* help is the other interfaces [1]:

Interface	What [1] specifies	Direct route	VISA resource (PyVISA [30], [31])	In 0.0.7
USB	“standard COM port”, 115200 8N1, CR; NI driver first	pyserial	`ASRL3::INSTR` (pyvisa-py uses pyserial)	**yes**
RS232	cable kit to a PC COM port, 115200 8N1, CR	pyserial	`ASRL1::INSTR`	yes — select that COM port
Ethernet	adapter, DHCP, “TCP/IP socket … on port 10001”, CR	Python `socket` (standard library)	`TCPIP0::<ip>::10001::SOCKET` (no extra package)	no
GPIB	adapter, “address 5”, CR	a GPIB driver (NI-488.2 or linux-gpib)	`GPIB0::5::INSTR` (NI-VISA, or pyvisa-py with gpib-ctypes / linux-gpib)	no
SPI	native commands only	—	—	no

The driver talks to its transport through four calls (`open`, `close`, `write`, `query`), which the simulator already implements a second time, so an Ethernet transport (standard-library socket) and a PyVISA transport (any VISA resource string, GPIB included, imported only when chosen) would each be one class. Neither was asked for, so neither is in 0.0.7 (§13).

0.0.10 does use PyVISA, for a different link: the optional power meter and frequency counter of the Calibration tab (§3.7), which are usually GPIB instruments. It is imported only when a meter is opened; the synthesizer link stays on pyserial.

---

3. SCPI coverage

Every command in [1, Table 5] is implemented, plus two native commands for the output level (§3.3). “Verified” means the formatted byte string was asserted against the vendor’s own worked example.

SCPI	Driver method	Range / argument	Verified
`FREQ` / `FREQ?`	`set_frequency` / `get_frequency`	integer mlHz	✔
`POW` / `POW?`	`set_power` / `get_power`	±XX.X dBm, 0.1 dB granularity, sign always written (`POW +12.0`)	✔
native `03xxxx` (Set Output Power)	`set_power(command_set="native")`, `native_power_command`	tenths of a dBm, 16-bit two’s complement, ASCII hex [1, Table 2a]	✔ (both [1] examples)
native `0D` (Get Power)	`get_power_native`, `decode_native_power`	4 ASCII-hex characters, dBm × 10 [1, Table 4]	— (no worked example in [1])
`OUTP:STAT[?]`	`set_rf_output` / `get_rf_output`	`ON`/`OFF`	✔
`OUTP:BLAN[?]`	`set_blanking` / `get_blanking`	`ON`/`OFF`	✔
`OUTP:ROSC:STAT[?]`	`set_reference_output` / `get_reference_output`	`ON`/`OFF`	✔

SCPI	Driver method	Range / argument	Verified
`ROSC:SOUR[?]`	`set_reference_source` / `get_reference_source`	`INT`/`EXT`	✔
`FREQ:LRSTAT[?]`	`set_lock_recovery` / `get_lock_recovery`	`ON`/`OFF`	✔
`PULM:STAT[?]`	`set_pulse_modulation`	`ON`/`OFF`	✔
`AM:STAT[?]`, `AM:DEPT[?]`	`set_am`, `set_am_depth`, `get_am_depth`	DAC 0–4095	✔
`FM:STAT[?]`, `FM:MODE[?]`, `FM:SENS[?]`	`set_fm`, `set_fm_mode`, `set_fm_sensitivity`	mode 1–4, DAC 0–4095	✔
`DIAG:CAL:REF:DAC[?]`	`set_reference_dac` / `get_reference_dac`	DAC 0–65535	✔
`*RST`, `*SAV`, `*RCL`	`reset`, `save_state`, `recall_state`	slots 1–2 / 0–2	✔
`*IDN?`	`identify`	five comma-separated fields	✔
`STAT?`	`get_status`	8-bit field, lock bits inverted	✔
`DIAG:MOD?`	`get_modulation_status`	6-bit field	✔
`DIAG:MEAS? 21`	`get_temperature`	°C	✔
`LIST:PVEC`, `LIST:PVEC:RUN`	`write_list_point`, `run_list_point`	point 1–32767	✔
`LIST:SETUP`, `LIST:STAR`, `LIST:STOP`	`setup_list`, `start_list`, `stop_list`	runs 0–32767	✔
`LIST:SAV`, `LIST:ERAS`	`save_list`, `erase_list`	—	✔
`SWE:FAST:FREQ:SETUP` / `:STAR`	`setup_fast_frequency_sweep` / `start_sweep`	≤ 32767 points	✔
`SWE:NORM:FREQ:SETUP` / `:STAR`	`setup_normal_frequency_sweep` / `start_sweep`	explicit step	✔
`SWE:FAST:POW:SETUP` / `:STAR`	`setup_fast_power_sweep` / `start_sweep`	≤ 500 points	✔
`SWE:NORM:POW:SETUP` / `:STAR`	`setup_normal_power_sweep` / `start_sweep`	explicit step	✔
`SWE:STOP`	`stop_sweep`	—	✔
`DIAG:BAUD[?]`	(exposed, debug port only)	baud	—

Of the IEEE 488.2 common commands [6] the QuickSyn implements only `*IDN?`, `*RST`, `*SAV` and `*RCL`. There is **no** `*CLS`, `*ESR?`, `*OPC`, `*STB?` or error queue, so command acceptance can be confirmed only by reading the affected parameter back. The application does exactly that after every reset and on demand from **Instrument → Read all settings**.

3.1 Mandatory settling delays

Applied automatically by the driver, from [1, Tables 2a/2b].

After	Wait
`*RST`	2 ms
`*SAV`	100 ms
`*RCL`	50 ms
`LIST:ERAS`	200 ms
`LIST:PVEC` with `,F` (flash)	300 ms
`LIST:PVEC` RAM only	100 µs
`LIST:SAV`	50 ms + 2.5 ms per point
`DIAG:CAL:REF:DAC`	a few seconds

After	Wait
`FREQ` while FM is on	1 ms

3.2 Interlocks enforced

- Lists and sweeps are refused while FM is on [1, Tables 2b/2c].
- Dwell times are quantised to the mandatory **5 µs** increment.
- Power is quantised to 0.1 dB, the command granularity.
- Frequency and power are range-checked against the detected model.
- The level can be set only on models with a power-control option (FSW, option 2); fixed-output models refuse `POW` and power sweeps (§3.3).
- A typed value outside the model's range is refused with a message and nothing is sent (§3.6).
- Every command is pre-flighted against the 64-byte buffer.
- The Modulation tab is hidden for models with no PULSE/AM/FM inputs.

3.3 Output power control

Why the level could not be changed in 0.0.6. Three faults in the application, one ambiguity in [1], and one property of the hardware:

#	0.0.6 behaviour	0.0.7
1	Typing a level and pressing Enter sent nothing; only *Set power* did (the same for frequency)	Enter in the frequency or power box applies it
2	*Set power* printed "commanded" without asking the instrument, so a unit that ignored `POW` looked as if it had obeyed	Every change is read back (`POW?`, `FREQ?`); a mismatch is shown in red and in the status bar, session log and SCPI console
3	Fixed-output models (FSL Lite, FSL mmW) showed an editable box that snapped back, and the mmW models offered their min/max output *specification* as if it were a control range	Power box and power sweeps disabled with the reason shown; the driver refuses `POW`
4	Only the SCPI form was available, written without a `+` sign for positive levels; [1, Table 5] gives `POW +/- XX.X` with only a negative example	Sign always written; *Command* selector: Auto (SCPI, then the native command if `POW?` shows no change), SCPI only, Native only
—	Hardware: power control is option 2 on the FSW [4], and [1] marks the native Set Output Power command "*When option is present". Without option 2 the output stays at the rated level whatever is sent	Cannot be fixed in software. *IDN? does not say whether option 2 is fitted (§9 #2), so it has to be found out on the unit (§13)

Ranges. From [4]; the entry box clamps to the detected model.

Model	Power control	Range with option 2	Resolution (nominal)	Without option 2
FSW-0010	option 2	−25 … +15 dBm	0.01 dB	+15 dBm fixed
FSW-0020	option 2	**−10 … +13 dBm**	0.1 dB	+13 dBm fixed
FSL-0010/0020/E020 (Lite)	none (option 8 only)	—	—	fixed: +15 / +10 / +10 dBm min
FSL-2740/5067/7682 (mmW)	none (options 3 and 8 only)	—	—	fixed: +18 / +17 / +11 dBm typ

Commands carry tenths of a dBm in both forms, so the entry is quantised to 0.1 dB on every model. Power accuracy is ±2.0 dB typical and “may change at lower power levels” [4].

Native Set Output Power [1, Table 2a]. For a level P in dBm,

$$N = \mathrm{round}(10\,P), \qquad W = N \bmod 2^{16} = \begin{cases} N, & N \geq 0, \\ 2^{16} + N, & N < 0, \end{cases}$$

and the command is the header `03` followed by W as four upper-case hexadecimal digits, sent as ASCII text and terminated with CR like every other command (“each hexadecimal character is one ASCII character” [1]). `native_power_command()` reproduces both worked examples in [1]:

P (dBm)	N	W	Command	Source
+12	120	120 = `0078`	`030078`	[1, Table 2a] example 2
−3	−30	65 506 = `FFE2`	`03FFE2`	[1, Table 2a] example 3
+13	130	130 = `0082`	`030082`	FSW-0020 upper limit
−10	−100	65 436 = `FF9C`	`03FF9C`	FSW-0020 lower limit

The native *Get Power* query `0D` [1, Table 4] returns four ASCII-hex characters, dBm × 10; *Read (0D)* on the Main tab decodes it with the same two’s-complement rule (an assumption: [1] gives no negative example for the query), as an independent check on `POW?`.

Read-back, and its limit. In *Auto*, *Set power* sends `POW +12.0`, then `POW?`; if the reply differs from the request by more than 0.05 dB it sends `030078` and asks `POW?` again. If the native command worked, the selector switches to *Native* for the rest of the session and says so. If neither did, the read-back turns red and the warning names option 2. `POW?` reports the level the firmware *holds*; it is not a measurement. A unit without option 2 could store the requested value, report it, and still put out its rated level, so the read-back can catch firmware that ignores the command but not missing hardware. Only a power meter or spectrum analyser at the RF output settles that (§13).

On hardware (0.0.8). The FSL-0020 of §10.1 accepted `POW 5.0`, `POW +0.0`, `POW -3.0` and `POW -10`; `POW?` and `0D` reported each level (`0D` replies `0032`, `0000`, `FFE2`, `FF9C`, which confirms the two’s-complement reading), yet a power meter showed no change, and NI’s own soft front panel could not change it either. The output does change with frequency, as an unlevelled source does. This is the blind spot described above, seen for real: the firmware stores the level, the hardware does not follow. It agrees with the datasheet, which lists only option 8 for the Lite models [4], and 0.0.7 already disabled the power box for the FSL-0020.

The unit’s option word (`0000017f`) nevertheless lists option 2, and NI’s calibration panel shows “1, 2, 3, 4, 5, 6, 7, 9”. The calibration manual offers one documented mechanism for “option present but no control”: power calibration can be written with the *Fixed Power Option*, in which case “control of power is disabled and the power is set to the specified fixed power” [3, §2-2]. Whether that, or simply missing levelling hardware, applies to this unit cannot be told from here. Two checks, in order of risk:

Check	What it shows	Risk
NI calibration panel, *Max/Min Pwr Sweep* with *Check Min (−25) and Max (+15)* [3, §3-3]	Whether min and max traces differ at all, i.e. whether the hardware can vary the level	None stated: a measurement sweep (power meter on GPIB, about 10 min)
Start Cal → *Write Cal to UUT* (variable power) → *Verify Cal*	Whether a variable-power calibration makes `POW` work	Write Cal to UUT goes to **volatile** memory and a power cycle restores the stored calibration [3, §2-2]; **do not press Store in Flash**
Ask NI with serial 1530200407	Whether this FSL-0020 was built with power control	None

[3] sets calibration frequencies and levels “based on the QuickSyn synthesizer model, FSW-0010 or FSW-0020”, so the panel may not support an FSL at all; stop if it refuses.

Console Send line. For bring-up, the SCPI console has a *Send* line: any SCPI or native command, sent as typed through the same port, 64-byte check and log as everything else. Enter sends it as a query when it ends in `?` or is a native query code (`01 02 04 07 0D 10 47 48 49` [1, Table 4]), otherwise as a write; *Write* and *Query* force either. It bypasses the interlocks and range checks, and the tabs are not refreshed — use *Instrument* ▸ *Read all settings* afterwards.

3.4 RF OUTPUT OFF

Three routes, all sending `OUTP:STAT OFF` and then `STAT?` to confirm:

Control	Where
Red **RF OUTPUT OFF** button	Main control tab, *Output and reference*
Red **RF OFF** button	*Safety* toolbar, visible from every tab
Instrument ▸ *RF output off (panic)*	menu, **Ctrl+Shift+O**

The status bar says “RF output OFF (confirmed by STAT?)”, or warns if `STAT?` still shows the output on. The controls stay enabled on every model. Off means muted, not disconnected: −65 dBm max on the FSW (“Power Mute”), a 60 dB minimum on/off ratio on the Lite models [4].

3.5 10 MHz reference: source, output and LEDs

Control	Commands (0.0.8)	Checked with
Reference source combo	`ROSC:SOUR INT/EXT`; native `0600/0601` [1, Table 2a] if `ROSC:SOUR?` disagrees	`ROSC:SOUR?`, `STAT?`, and again 1.5 s later for the lock
10 MHz reference output switch	`OUTP:ROSC:STAT ON/OFF`; native `0800/0801` if the output did not follow. Once the native command has worked where the SCPI one did not, it is sent directly for the rest of the session (0.0.9). On a unit that reports the state nowhere, `OUTP:ROSC:STAT` alone after the first double failure (0.0.10)	`OUTP:ROSC:STAT?` when the unit answers it (trusted over bit 5, 0.0.10); otherwise `STAT?` bit 5 until it is seen not to follow

Two LEDs in the status box show which reference is **in use**: *Internal 10 MHz reference IN USE*, or *External 10 MHz reference IN USE* when `ROSC:SOUR?` says EXT, a signal is present at REF IN (`STAT?` bit 0) and the reference is locked. External selected with no signal shows *NO SIGNAL at REF IN* in red; selected but unlocked shows *NOT LOCKED*. The existing *External reference detected* LED is presence only: on the FSL-0020 it was lit while the internal reference was in use.

The 0.0.7 crash on choosing External (“ ‘str’ object has no attribute ‘value’ ”) came from PySide6 returning a `StrEnum` stored as combo-box data as a plain `str`; `IntEnum` data survives. The driver now converts every enum argument itself.

The FSL-0020’s REF OUT does switch off: confirmed by the user with NI’s soft front panel and a power meter. NI’s panel uses the native command set (its command window lists entries such as `Pulse Mod [2] | 09 00 |` and `FM Mode [2] | 0B 04 |`, the native headers of [1, Table 2a]). From this program, `OUTP:ROSC:STAT OFF` was sent three times and `STAT?` kept bit 5 set, so the SCPI form is the part that does not act on that unit (firmware 0008), or does not update the status bit; which of the two is open until REF OUT is measured while this program switches it.

0.0.9 therefore:

- checks `STAT?` bit 5 and, only if the unit answers it, `OUTP:ROSC:STAT?`. That query was never seen to answer on the FSL-0020, and in 0.0.8 a time-out there would have stopped the switch before the native fallback and failed every *Read all settings*, and with it auto-read. One failure now marks it unsupported for the session;
- falls back to the native `0800/0801` if the output did not follow, and once that works, sends it directly for the rest of the session (the status bar says so), as the power command already does;
- if neither form changes `STAT?` bit 5, warns and asks for a check of REF OUT on a power meter or counter: if REF OUT did switch, only the status bit is wrong.

Settled on 2026-10-14 (§10.2): only the status bit is wrong. With a power meter on REF OUT, `OUTP:ROSC:STAT OFF` and `ON` sent from this program switch the 10 MHz output off and on, and `STAT?` bit 5 stays set throughout. 0.0.9 would therefore have warned falsely and sent the native `08xx` as well. 0.0.10:

Situation	What 0.0.10 does	Shown
`OUTP:ROSC:STAT?` answers	The query decides; bit 5 is not used for the output. A bit 5 that disagrees marks the bit unreliable for the session (status-bar note, once)	LED and checkbox from the query: *Ref output off (OUTP:ROSC:STAT?)*
No query, bit 5 follows	As 0.0.9	LED and checkbox from bit 5
No query, bit 5 follows neither `OUTP:ROSC:STAT` nor `08xx` (the first double failure)	One warning explaining that on the FSL-0020 REF OUT switches while bit 5 stays set; from then on `OUTP:ROSC:STAT` alone is sent, with no fallback and no warning	Grey LED *Ref output: not reported (last sent OFF)*; the checkbox keeps what was sent and is no longer overwritten from bit 5 by *Read all* or auto-read

The simulator option `refout_status_bit_follows=False` holds bit 5 set, as on the FSL-0020; `check_calibration.py` covers both rows that differ from 0.0.9.

3.6 Typing values

Frequencies below 1 GHz could not be typed in 0.0.7. With nine decimals shown (`1.000000000 GHz`), the stock spin box refuses any digit typed after the point, so editing the text could not produce 0.8; text it judged incomplete or out of range was silently put back when focus moved, and *Set frequency* then re-sent the old value. Reproduced with Qt 6.12:

Keys, starting from `1.000000000 GHz`	0.0.7	0.0.8
Home, Delete, `0.800`, Enter	`0.000000000` → reverted → **1 GHz re-sent**	0.8 GHz
End, Backspace ×15, `0.800`, Enter or *Set*	nothing deleted → 1 GHz re-sent	0.8 GHz
Home, Delete ×2, `0.8`, Enter	reverted → 1 GHz re-sent	0.8 GHz
Select all, `0.800`, Enter	0.8 GHz	0.8 GHz
Select all, `0.8123456789123`, Enter	refused (too many decimals)	0.812345679 GHz (rounded to 1 mHz)
Select all, `0.1` (below range), Enter or *Set*	reverted, old value sent	**refused with a message, nothing sent**

Every numeric box is now an `EntrySpinBox`: any number may be typed, extra decimals are rounded on commit, End and Backspace work on the number rather than the unit, typing . just before the existing

point steps over it, and a value that is not a number in range is reported (“ ‘0.1 GHz’ is not a number from 0.2 to 20 GHz; the box keeps … and nothing was sent”) instead of being replaced in silence. The arrows and wheel work as before.

3.7 Calibration (program-side)

NI’s calibration soft front panel [3] calibrates output power and the 10 MHz OCXO, but only with GPIB instruments it knows: an Agilent E4418B power meter “or exact SCPI-command equivalent” with an E4413A sensor and a 6 dB pad, and a Pendulum CNT-90, Anritsu MF2412B or Phase Matrix 346A counter, and it sets its power calibration points “based on the QuickSyn synthesizer model, FSW-0010 or FSW-0020” [3]. The **Calibration** tab does the same measurements with readings typed in or read from *any* SCPI meter through PyVISA [30], [31] (GPIB, USB or LAN), and keeps the results in this program.

What is written to the synthesizer, and what is not.

NI panel step [3]	Documented command?	This program
Ref. Cal: *Start Ref Cal* — tune until RF OUT is within ±20 Hz at 10 GHz; “stored in volatile memory”	**Yes**: `DIAG:CAL:REF:DAC 0–65535`, “a few seconds for hardware to update” [1, Table 5]	Same tuning, same command, same ±20 Hz default; the value can be re-sent on every connect
Ref. Cal: *Write Ref Cal to UUT* — permanent	No	Not sent. The value is saved per unit in this program instead
Power: *Write Cal to UUT* (volatile) and *Store in Flash* (“will overwrite previous calibration data”)	No	**Not sent.** Guessing an unpublished write could destroy the factory calibration, and a power table cannot give an FSL level control it lacks (§3.3). The table is kept in this program
Power: *Max/Min Pwr Sweep*, *Verify Cal*	Measurement only	The *Power vs frequency* page does both: a point list, meter readings, the table and a plot against the datasheet

The vendor describes the output power control as “an open-loop method” with temperature compensation [37]. An open-loop source can only be as flat as its calibration, so a table measured at RF OUT is the right object to keep; what it cannot do is change the hardware’s own correction.

Power vs frequency.

1. *Points*: start, stop and step frequency, and on power-control models a list of commanded levels (for example `-10, 0, 13`). Points outside the model’s range are dropped and counted.
2. *Path loss* between RF OUT and the sensor: none, a constant, a CSV table (frequency and loss in dB; units from the header or by magnitude; a column that is all negative is read as S_{21} in dB and negated) or a Touchstone `.s2p` [35] (v1 and v2, `MA`/`DB`/`RI`, both `[Two-Port Data Order]` values; noise data ignored). A file is never extrapolated: a point outside it is refused.
3. *Set synthesizer to this point* sends `FREQ` (and `POW` on power-control models) and RF on, and reads the status back. Type the meter reading and press **Enter**: it is recorded and, with *Auto-advance*, the next point is set. *Read power meter* reads it instead; *Run all points with the power meter* does the whole list with a settle time per point, and *Stop* ends it.
4. *Save and make active for this unit* writes the table (§ files below) and makes it the unit’s active table, loaded again on every connect.

The level at RF OUT is the reading plus the path loss,

$$P_{\mathrm{out}}(f) = P_{\mathrm{meter}}(f) + L(f), \qquad L(f) = -20\log_{10}|S_{21}(f)|\ \mathrm{dB},$$

with $|S_{21}|$ taken directly (`MA`), as $\sqrt{a^2+b^2}$ from real and imaginary parts (`RI`), or $L = -S_{21,\mathrm{dB}}$ (`DB`). The table and the plot show P_{out}, and against the datasheet minimum for fixed-output models (+15 dBm FSL-0010, +10 dBm FSL-0020/E020 [4]). Between calibrated frequencies $f_i < f < f_{i+1}$ the level is interpolated linearly in dB,

$$P_{\mathrm{out}}(f) = P_i + (P_{i+1} - P_i)\,\frac{f - f_i}{f_{i+1} - f_i},$$

and outside $[f_1, f_n]$ nothing is reported. With *Show expected level at RF OUT* ticked, the Main tab shows this value under the power read-back (*At RF OUT (power cal)*), updated with every frequency change.

Level correction (power-control models). With several commanded levels P_s measured, the table holds the error $e(f, P_s) = P_{\mathrm{out}}(f, P_s) - P_s$, interpolated in frequency as above and linearly in P_s (held constant beyond the lowest and highest level). To put a target P_t at RF OUT, the level to command solves $P_s + e(f, P_s) = P_t$, by the fixed-point iteration

$$P_s^{(0)} = P_t, \qquad P_s^{(k+1)} = \mathrm{clamp}\Big(P_t - e\big(f, P_s^{(k)}\big),\ P_{\min},\ P_{\max}\Big),$$

stopped when two iterates differ by less than 0.005 dB and then quantised to the 0.1 dB of the command set (§3.3). It converges whenever $|\partial e/\partial P_s| < 1$ (a contraction), i.e. whenever the output rises with the commanded level at all. With *Correct the commanded level* ticked, *Set power* on the Main tab sends the corrected level, the read-back says so ("set +2.0 dBm for +0.0 dBm at RF OUT"), and a frequency change re-applies it. Sweeps and list mode are **not** corrected. On fixed-output models the box is disabled: there is no level to correct.

Reference (10 MHz OCXO). The method of [3] Step 4: a counter on RF OUT, referenced to a 10 MHz standard. *Start* reads `DIAG:CAL:REF:DAC?` (kept as the original), selects the internal reference, sets the counter frequency (10 GHz by default) and switches RF on. Each measured frequency is recorded (typed, or *Read counter*) and the next DAC word follows from the secant method on the two latest distinct points,

$$D_{k+1} = D_k - (f_k - f_{\mathrm{t}})\,\frac{D_k - D_{k-1}}{f_k - f_{k-1}},$$

after a first trial step (500 counts by default, only to measure the slope), limited to 8000 counts per step and to 0–65535. The default tolerance, ±20 Hz at 10 GHz [3], is

$$\frac{20\ \mathrm{Hz}}{10\ \mathrm{GHz}} = 2 \times 10^{-9}, \quad \text{i.e. } \pm 0.02\ \mathrm{Hz} \text{ at } 10\ \mathrm{MHz}.$$

Send next DAC value (automatic after each recording by default) writes it and waits the documented few seconds. *Restore original DAC* undoes the session; *Save as this unit's reference calibration* keeps the best iteration and, with *Apply the saved value on connect*, sends it every time this program connects (the status bar says so). Whether a value written with `DIAG:CAL:REF:DAC` survives a power cycle is not documented; [3] says the panel's own tuning value is volatile until *Write Ref Cal to UUT*.

Meters (optional). Any instrument whose query returns one number.

Field	Power meter default	Counter default	Note
VISA resource	—	—	e.g. `GPIB0::13::INSTR`, `USB0::…::INSTR`, `TCPIP0::host::inst0::INSTR`; *List resources* shows what the VISA library sees

Field	Power meter default	Counter default	Note
Back end	default VISA library	default VISA library	NI-VISA or Keysight IO Libraries (needed for most GPIB adapters), or `@py` (pyvisa-py)
Query	`MEAS?`	`MEAS:FREQ?`	common SCPI forms; check them in the instrument's manual
Pre-command	`SENS:FREQ {freq_hz:.0f}`	—	written before each reading; `{freq_hz}` / `{freq_ghz}` are replaced by the point's frequency, so the sensor applies its cal factor
Timeout	10 000 ms	10 000 ms	

Connect and identify sends `*IDN?`, which is stored in the table. Without PyVISA the page says how to install it (`mamba install -c conda-forge pyvisa pyvisa-py`) and typed readings still work.

Files. Under `~/quicksyn_cal/<model>_<serial>/` (set `QUICKSYN_CAL_DIR` to move it); nothing is created until something is saved.

File	Content
`power_cal_<YYYYmmdd_HHMMSS>.json` / `.csv`	Each saved table: unit, operator, meter `*IDN?`, path-loss description, every point (frequency, commanded level, reading, path loss, level at RF OUT, time, source)
`active_power_cal.json`	The table loaded on connect
`ref_cal.json`	Saved DAC word, original word, target, residual error, apply-on-connect flag
`ref_cal_history.csv`	One line per reference save

Export CSV… and *Export .vszh5…* write a table anywhere; the `.vszh5` has one page (level at RF OUT against frequency, one curve per commanded level, the datasheet minimum dashed) and opens in Veusz.

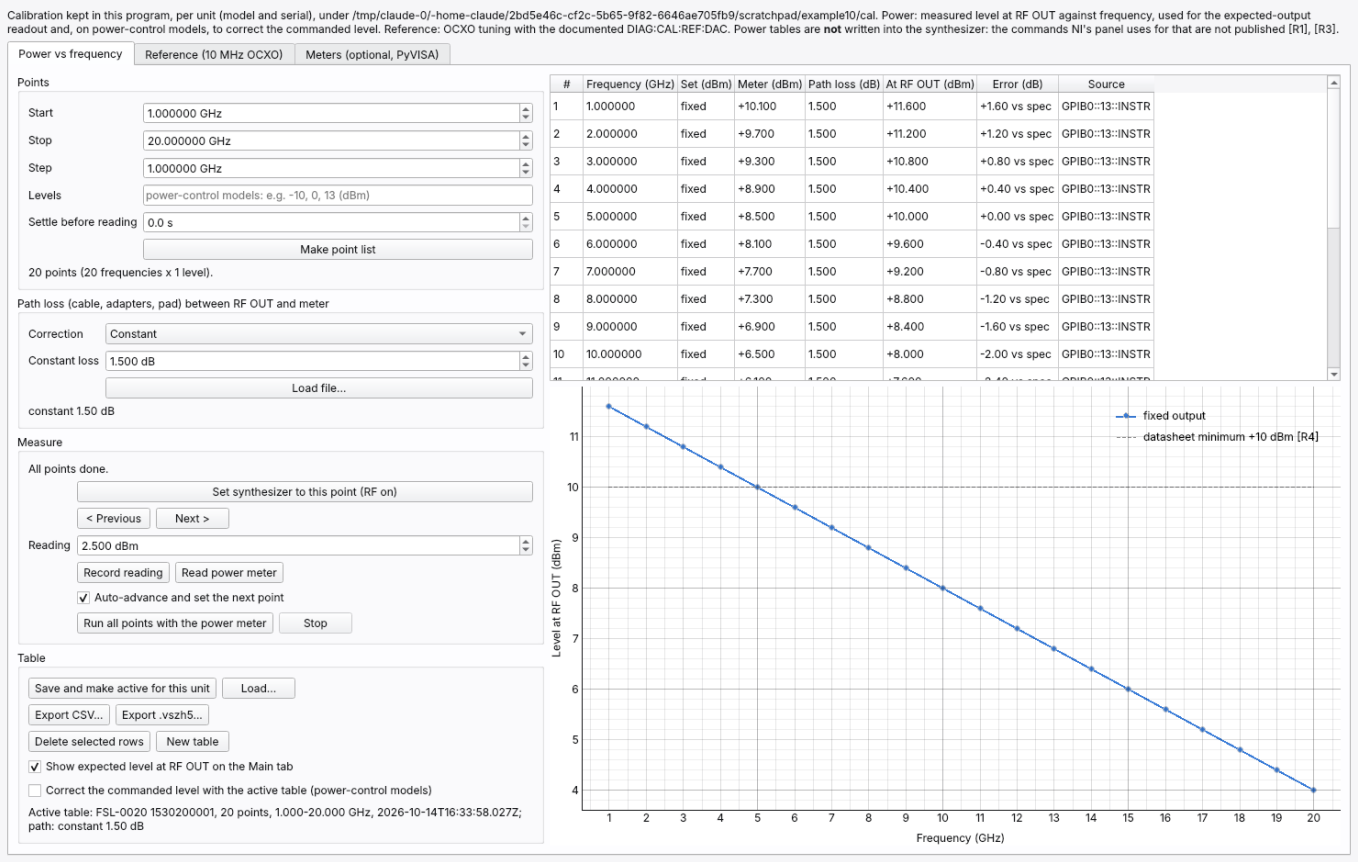


Figure 1: Calibration tab, *Power vs frequency* page, after a **simulated** 20-point run on the FSL-0020 simulator (synthetic level $12 - 0.4\,f$ /GHz dBm, 1.5 dB constant path loss). Not a measurement.

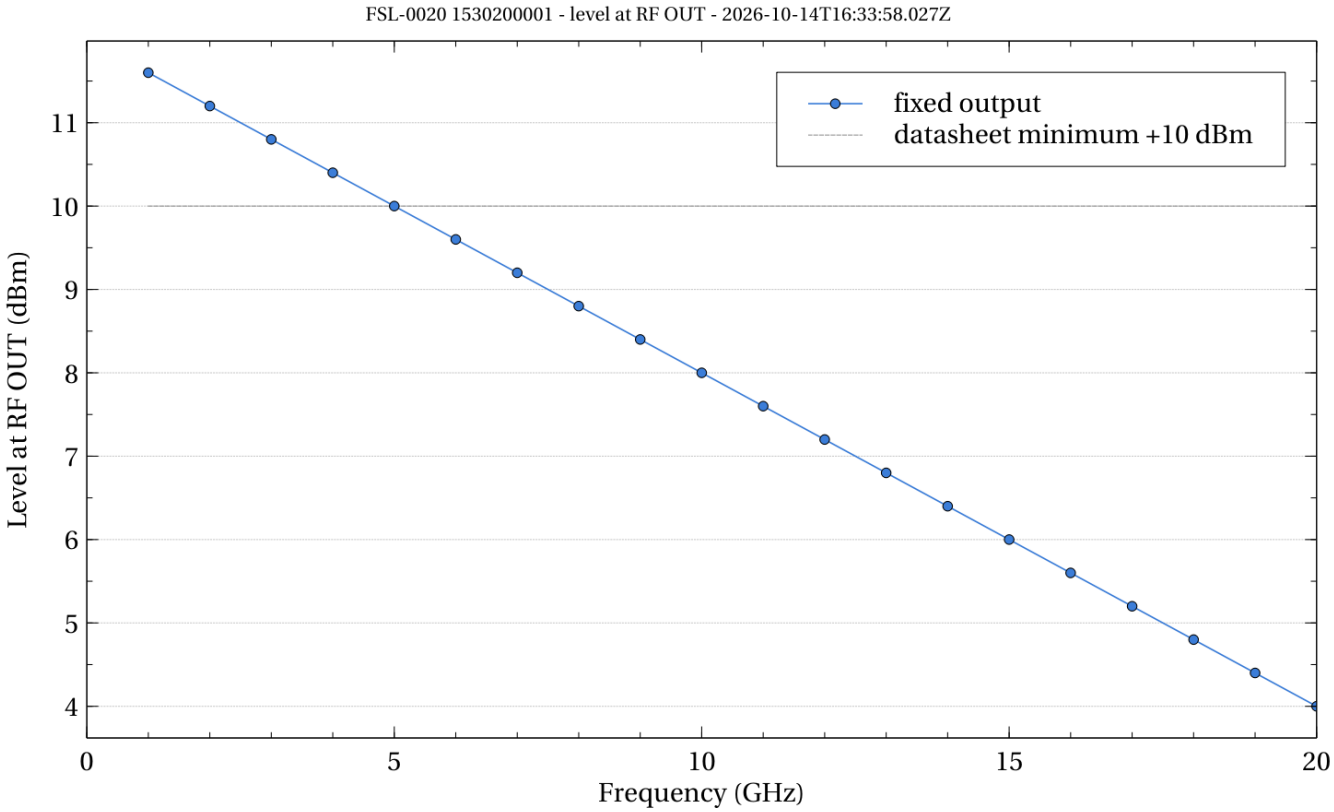


Figure 2: The same simulated table exported as `.vszh5` and rendered by Veusz 4.2.1.

4. The 64-byte buffer, and why frequencies are encoded two ways

Millihertz is the documented default unit for `FREQ` [1, Table 5], and a standalone `FREQ` is sent that way because it is exact and always fits. But a 20 GHz carrier is 14 characters in millihertz, and

```
SWE:FAST:FREQ:SETUP 2000000000000,10000000000000,80,2.0dBm,1s,10,0,0
```

is **69 bytes with its terminator** — five over the device buffer. This is why every compound example in [1] writes frequencies with a `GHz` suffix. `format_frequency()` therefore returns the shortest *exact* spelling among millihertz, `KHz`, `MHz` and `GHz`:

Requested	Encoded	Why
2.000000000 GHz	`2GHz`	4 bytes beats 13
2.200000000 GHz	`2.2GHz`	matches the `FREQ 2.2GHz` example in [1]
0.100000000 GHz	`0.1GHz`	6 bytes beats `100MHz`, ties go to GHz
9.111222333 GHz	`9111222333000`	13 bytes beats `9.111222333GHz` at 14

The same sweep now encodes to 51 bytes. A command that still will not fit — three full-precision frequencies in a normal sweep, say — is refused *before* transmission with advice on which parameter to round, rather than being silently truncated by the device.

---

5. Logging

Each session creates `<log_root>/<YYYYmmdd_HHMMSS>_<model>_<opts>/`:

Artefact	Contents
`session_metadata.json`	Identity, capability row, transport settings, sampler configuration, record dtype, versions
`session.log`	Human-readable event log
`transactions.csv`	`seq, iso_utc, t_unix, direction, command, response, latency_ms, ok, error` — every byte exchanged
`state_events.csv`	`seq, iso_utc, t_unix, parameter, value, source` — every commanded change of generated-signal state
`telemetry.csv`	25 columns: timestamps, frequency, power, temperature, decoded status and modulation bits, latency, instantaneous rate
`telemetry_chunk_####.pkl.gz`	gzip-pickled NumPy blocks of the same records, 52 bytes each (written via `.part` + rename)
`quicksyn_session.vszh5`	Veusz HDF5 document: all data and plot pages (§6) — written by the controller at stop, on demand, or every N minutes

The log root is set on the *Monitor and log* tab and defaults to `quicksyn_logs` in the home folder — on the hardware PC, `C:\Users\wwallace\quicksyn_logs\20261012_140204_FSL-0020_0000017f\` (the path is also the second line of `session.log`). NI's QuickSyn soft front panel keeps no log file: its command window only displays the commands of the session [2]; only *Save List to File* (List Mode) and the calibration panel's *Save Data to File* write files, to wherever their file dialog points [2], [3].

`transactions.csv` is the audit trail of *what was sent*; `state_events.csv` is the readable record of *what was generated*; `telemetry.csv` is *what the instrument reported back*, at the sample rate.

5.1 Sample rate

The requested period is

$$\tau_0 = \frac{1}{f_s}$$

and each sample costs one serial round trip per enabled field. With N_q queries per sample and mean round-trip time t_q the ceiling is

$$f_{s,\max} = \frac{1}{N_q\, t_q}$$

Polling frequency, power, status, modulation and temperature gives $N_q = 6$ — temperature counts twice while the conservative double-issue rule is enabled — so at $t_q \approx 8$ ms,

$$f_{s,\max} \approx \frac{1}{6 \times 8 \times 10^{-3}} \approx 21 \text{ Hz}$$

The Monitor tab shows this budget live and turns it into a warning when the requested rate exceeds it; the **achieved** rate and the serial duty cycle are measured once per second, not estimated. Temperature decimation (every *N*th sample, default 5) buys the bandwidth back, since the internal temperature moves on a thermal time constant of minutes.

Scheduling is absolute: the deadline for sample k is computed from the acquisition start time as $t_0 + k\,\tau_0$, so jitter in one iteration does not accumulate into drift.

5.2 Memory discipline

Mechanism	Effect
Preallocated NumPy ring buffer	Fixed RAM: `capacity × 52` bytes, allocated once (60 000 records ≈ 3.1 MB)
Spill on overflow	A full block is handed to the writer thread and the index resets, so steady-state RAM is flat however long the run
Queue-decoupled writer thread	All disk I/O leaves the sampler thread; a slow filesystem cannot distort the sample interval
Flush RAM to disk now	Spills the resident block on demand
Live accounting	Ring fill, buffer RAM, process RSS, writer queue depth and chunk count on the Monitor tab

5.3 Parallelism and GPU

Device I/O cannot be parallelised — the instrument is one half-duplex endpoint behind a single reentrant lock. What *is* parallel: the sampler thread, the writer thread, and the GUI, plus process-pool chunk loading and optional CuPy reductions in the companion script. The controller uses no GPU on purpose: at a ~20 Hz ceiling the host-device transfer would cost more than the arithmetic it replaces.

5.4 Live SCPI console

View ▸ SCPI console (**Ctrl+Shift+C**) shows every exchange with the instrument as it happens, in a pane docked under the tabs with room for about ten lines. It can be resized with the splitter, undocked, moved to any edge or closed, and opens again from the View menu. To make that room, the tab pages now scroll when the window is shorter than a page: in 0.0.4 the tallest page (Monitor and log) forced a minimum window height of 981 px, which would not fit a small laptop screen. The window opens at 1180 × 960 px, or smaller if the screen is.

```
19:45:19.229 GUI  → POW -3.0  (write, 0.6 ms)
19:45:19.231 GUI  → FREQ?
                  ← 12500000000000  (1.6 ms)
19:45:21.652 GUI  → FREQ 9876543210000  (write, 0.6 ms)
19:45:21.653 GUI  → POW?
                  ✖ empty response (timeout)  (500.0 ms)
19:45:22.643 POLL → FREQ?
                  ← 9876543210000  (1.6 ms)
```

(Lines copied from the 0.0.6 console in a simulated session; the timeout was injected. The `POLL` line is visible because *Show telemetry polls* was ticked.) After a long pause with polls shown, the redraw starts with one grey line, for example after 200 200 exchanges:

```
⋯ 197700 earlier exchanges not shown here (the console keeps the newest 5000 lines); the session's
 ↪  transactions.csv holds every exchange while logging
```

Each line starts with the local time to the millisecond and the source. A command that expects no reply is one line with its write time; a query is the command and, under it, the reply or the error.

Element	Meaning	Colour
→	Command sent, exactly as transmitted (terminator not shown)	blue

Element	Meaning	Colour
←	Reply received, with the round-trip latency	green
✖	Timeout or I/O error, with the reason	red
`GUI` / `POLL`	Issued by you (any tab or menu) / by the telemetry sampler	grey / amber

The colours are mid-tones chosen to stay legible in the light and dark themes.

Control	Effect
Show telemetry polls	Off by default: at 10–20 Hz the sampler's `FREQ?`/`POW?`/`STAT?` traffic would bury your own commands. Polls are still counted and still logged to `transactions.csv`; ticking the box redraws the retained history with them
Filter	Shows only exchanges whose command, reply or error contains the text, e.g. `POW` or `LIST`. Applied when typing pauses for 250 ms, or at once on **Enter**. While paused it searches the frozen view
Pause / Resume	Freezes the view; exchanges keep arriving and the newest appear on **Resume**. A pause of any length costs no extra memory, and Resume draws at most the 5000 lines the view can hold, with a grey … line counting anything older
Auto-scroll	Follows the newest line while the view is at the bottom
Copy / Save… / Clear	Copy the view, save it as a `.txt` file, forget everything shown

The status line counts exchanges, polls and errors since the last clear.

How it is built. The driver reports every transaction through a Qt signal; exchanges made on the sampler thread cross to the GUI thread as queued signals (the slot runs when control returns to the GUI thread's event loop [24]), so no widget is touched from a worker thread. Receiving an exchange only files it, in constant time. A 150 ms timer draws whatever is new, so a 20 Hz poll stream costs about seven redraws per second, not one per exchange. Lines go in through a `QTextCursor` with one character format per colour [25], measured at 11–21 µs per line against 142–186 µs for `appendHtml` (5000–10 000 lines); that is also why replies line up exactly under their commands, where HTML had collapsed the space after `GUI`.

Bounds, whatever the session length and however long the view is paused:

Bound	Value	Why
Lines in the view	5000 (oldest removed first by `maximumBlockCount` [23])	The text widget's own memory and layout cost
Exchanges kept for re-filtering	20 000 **per source** (GUI and POLL)	Polls can never push your own commands out of what the filter searches
Lines drawn per redraw	≤ 5000, the rest summarised in one … line	Resume after any pause, or a filter change, blocks the window for well under a second

Bound	Value	Why
Memory while paused	measured 5.3 MB for 200 200 exchanges (0.0.5: 45.5 MB, growing without limit)	Nothing is queued: Resume draws from the two stores

0.0.5 queued every exchange that arrived during a pause and drew all of them on Resume. At the default 2 Hz sample rate (3.4 queries per sample) 200 000 exchanges is about 8 hours of pause; at the nominal 50 Hz maximum, about 20 minutes. Measured on the same simulated window:

200 200 exchanges while paused	0.0.5	0.0.6
Python memory held (tracemalloc)	45.5 MB (unbounded)	5.3 MB (bounded)
Resume, polls hidden	0.16 s	0.01 s
Resume, polls shown	**62 s** with the window frozen	0.07 s
Typing three filter characters, full store	4.7 s of redraws	one redraw after the typing pause
Command / reply arrow columns	`GUI` 17 / 18 (one off), `POLL` 18 / 18	18 / 18 for both

The console is a live view, **not** the record — `transactions.csv` (§5) remains the complete audit trail of every byte exchanged, and the … line says so.

From 0.0.7 the console also has a *Send* line for typing commands (§3.3), and warnings from the controller (a level that did not change, an RF OFF that `STAT?` does not confirm) appear in it as red `NOTE !` lines.

5.5 Auto-read of all settings

Auto-read all settings every N s (Main tab, under *Read all settings from instrument*; 1–3600 s, default 5 s) repeats the full read-back on a timer while connected: frequency, level, `STAT?`, temperature, reference source, `OUTP:ROSC:STAT?` (where answered) and modulation. A box being typed in is not overwritten. Three failed reads in a row stop it with a warning, and it stops on disconnect. It runs on the GUI thread and shares the port lock with the telemetry sampler, so each read costs about nine exchanges; at 5 s that is under 2 exchanges per second. It updates the display only; the telemetry log is still the sampler's.

6. Veusz documents, reduction and plots

```
python quicksyn_veusz_export_v0_0_10.py <session_dir>
    --time-axis A      local (default), utc or elapsed
    --no-render        write the .vszh5 but skip the PDF/PNG render
    --png-pages P      first (default, the overview), all or none
    --veusz-exe PATH   Veusz program for rendering (default: search)
    --render-method M  auto (default), program or inprocess
    --no-compress      store .vszh5 datasets uncompressed
    --no-veusz         statistics only: no .vszh5, no plots
    --no-gpu           force the NumPy host path
    --workers N        process-pool size (default: CPUs this process may use)
    --dpi N            PNG resolution (default 150)
    --no-latex         do not write or compile the LuaLaTeX report
    --lualatex PATH    lualatex to use (default: search, see section 6.7)
    --version, -V      print version, copyright and licence, then exit
```

Writes `reduction_summary.json`, `reduction_summary.md` (Obsidian-ready, LaTeX equations), `telemetry_reduced.npz`, and **`quicksyn_session.vszh5`**, a native Veusz HDF5 document written directly with `h5py` — **Veusz is not needed to create it**. Open it in Veusz (File ▸ Open) to restyle the plots. When Veusz is installed, `quicksyn_session.pdf` (every page, with a bookmark hierarchy) and page PNGs are rendered *from* the `.vszh5`, which also proves it opens, and the LuaLaTeX report `quicksyn_session_report.tex` is written and, when `lualatex` is found, compiled to `quicksyn_session_report.pdf` (§6.7). No text `.vsz` is ever written. The controller produces the same files itself (§6.4).

6.1 Why `.vszh5` and not `.vsz`

A text `.vsz` stores embedded data with `fmt='%e'` [18], i.e. as $m \times 10^k$ with $1 \le m < 10$ printed to six decimals: seven significant digits. The exporter stores the carrier in GHz, so with $k = \lfloor \log_{10} f_{\mathrm{GHz}} \rfloor$ the quantisation step and the worst-case rounding error are

$$q = 10^{\,k-6}\ \mathrm{GHz}, \qquad |\varepsilon| \le \frac{q}{2}, \qquad \frac{|\varepsilon|}{f} \le \frac{q/2}{10^k} = 5 \times 10^{-7}.$$

Carrier	k	Step q	Worst-case error
1 to < 10 GHz	0	1 kHz	500 Hz
10 to < 100 GHz	1	10 kHz	5 kHz

Measured on 200 000 random carriers per decade: maximum 500.0 Hz and 5000.0 Hz, as predicted. Examples: 9.876543210 GHz came back 210 Hz off, 11.123456789 GHz 3211 Hz off, and two carriers 1 mHz apart merged. The `.vszh5` stores IEEE-754 float64 verbatim (53-bit significand, $\approx 1.1 \times 10^{-16}$ relative), so every value round-trips bit-exact.

6.2 The writer: `vszh5_writer.py` (from `isp_speed_monitor.py`)

The Veusz file creation is the method from the author's `isp_speed_monitor.py` 1.3.0 [21], moved into its own module so both the controller and the export script use one implementation, and so it can be reused in other programs. Your call signatures are kept.

Item	Origin	What it does
`VeuszWriter`	`VeuszWriter`	Opens `<file>.part`, writes `/Veusz` with `vsz_format = 1`, typed datasets (`dataset_1d` with `serr`, plus `perr`/`nerr`; `dataset_date`; `dataset_text`) and tags; `finish()` stores the script and renames atomically
`VszDoc`	`VszDoc`	Builds the Veusz command script: `add`, `to`, `up`, `set` (paths like `x/label`)
`vz_text()`	`vz_text()`	Escapes label text — now identical to Veusz `latexEscape`
`epoch_to_veusz()`, `local_utc_offsets()`	`epoch_to_veusz()`	Unix seconds → Veusz dates (s since 2009-01-01); per-sample UTC offset, DST-aware
`find_veusz_exe()`	`find_veusz_exe()`	`QUICKSYN_VEUSZ` / `VEUSZ_EXE`, PATH, then the usual install folders

Item	Origin	What it does
`render_pages()`	`tests/render_in_veusz.sh`	Drives the Veusz program in `--listen` mode with `loadHDF5Doc`; PDF of every page and per-page PNGs; falls back to the Veusz Python package when the program is absent or fails; bookmarks the PDF
`add_pdf_bookmarks()`	new in 0.0.6	Writes a bookmark hierarchy and document information into a PDF with pypdf, atomically, and opens the bookmark pane

Changes from 1.3.0, each found by testing against real Veusz:

Change	Reason
`vz_text` maps `\` to `{\backslash}` and escapes `[ ]`	In Veusz text `\\` is a **line break** [18, `textrender.py`], so the 1.3.0 escape split a label at any backslash (a Windows path in a title)
Atomic `.part` write, `abort()` and a context manager built into the class	An exception mid-write discards the partial file; the previous complete file stays
Optional gzip + shuffle	8-hour sessions; Veusz reads compressed datasets transparently
`render_pages` waits for a done marker, then ends Veusz	Veusz 4.2.1 does **not exit** on `Quit()` in listen mode (3.6.2 does); `render_in_veusz.sh` would sit until its `timeout 180`
Paths in the render script go through `repr()`	Windows back-slashes in an interpolated path would become escape sequences
0.0.5: the PNG path is bound on its own line, outside the `exec()` string	0.0.4 put it inside the `exec()` string, where it was unescaped a second time: `C:\Users\…` failed to compile, `C:\data\new` gained a newline, and a quote in the path broke the string. Found while fixing the same code in the speed monitor
0.0.5: `method="auto"` falls back to the Python package when the program *fails*, not only when it is missing	A Veusz on the PATH can be present but broken: Ubuntu's `/usr/bin/veusz` starts `/usr/bin/python3`, and when that has been switched to another Python than the one the package was built for (3.13 instead of 3.12 in the test machine) it stops with `No module named 'PyQt5.sip'`. Both errors are reported if the fallback fails too
0.0.5: a program that exits before reading its script, or cannot be started, is a failed render	It raised `BrokenPipeError` or `OSError` instead of `RuntimeError`, which skipped the fallback; a race, so it only showed up sometimes
0.0.6: PDFs are exported with `pdfdpi=72`	Veusz 4.2.1 drew every PDF page in the top-left 48 % of the page (§6.5). The PNGs were always right, and the 0.0.4 checks inspected PNGs, so it went unnoticed until the PDFs were bookmarked and included in the LaTeX report
0.0.6: the PDF gets a bookmark hierarchy	Session ▸ Overview, Time series (one entry per quantity), Stability (§6.7)
0.0.6: a re-render reports its files	0.0.5 compared against the folder listing taken before the render, so rendering a second time into the same folder returned nothing and the controller showed no plots

6.3 What the session document contains

Datasets	Tag	Contents
`raw/time_utc`, `raw/time_local`	raw	Veusz dates of every sample (local follows DST)
`raw/elapsed_s`, `raw/epoch_s`, `raw/utc_offset_s`, `raw/index`	raw	Time bases and sample index
`raw/freq_hz`, `raw/freq_ghz`, `raw/power_dbm`, `raw/temp_c`	raw	Reported carrier, level, internal temperature
`raw/latency_ms`, `raw/interval_ms`, `raw/rate_hz`	raw	I/O latency, sample interval, requested rate
`raw/status_word`, `raw/modulation_word`, `raw/rf_on`, `raw/rf_unlocked`, `raw/ref_unlocked`, `raw/voltage_error`	raw	`STAT?` / `DIAG:MOD?` words and decoded flags
`events/*` (times, `parameter`, `value`, `source`, `freq_ghz`, `power_dbm`)	events	Every commanded change from `state_events.csv`
`stability/tau_s`, `stability/adev`	stability	Overlapping ADEV of the reported carrier
`meta/session`, `meta/session_metadata_json`, `meta/reduction_summary_json`, `meta/generator`	meta	Provenance

Page	Contents
`page_overview`	3 × 2 grid: carrier, level, temperature, latency, sample interval, ADEV
`page_carrier`, `page_level`	Reported trace with **commanded values overlaid as markers**
`page_thermal`, `page_latency`, `page_interval`	One quantity per page against the chosen time axis
`page_stability`	Log–log ADEV

6.4 From the controller

The controller writes the `.vszh5` itself — no separate step:

Trigger	Where
Logging stops (default on)	Monitor tab ▸ Veusz document ▸ *Write when logging stops*
On demand, during or after logging	*Write .vszh5 now*, Veusz menu, **Ctrl+E**
Every N minutes while logging	Monitor tab ▸ *Refresh while logging* (0 = off)
PDF/PNG and the LuaLaTeX report as well	*Also render PDF/PNG with Veusz*, or Veusz ▸ *Write .vszh5 and render*; the status bar names the plots and the report
View	*Open in Veusz* (uses `find_veusz_exe`, else the desktop file association)

The export runs the companion script **as a separate process**: the GUI's PySide6 never shares a process with the Qt binding Veusz uses, a long session never stalls the window, and the export started by closing the window runs detached and finishes after the window is gone. While logging, the RAM

ring is spilled first and the writer thread allowed to commit it, so the document holds every sample up to that moment; telemetry chunks are now written to `.part` and renamed, so a concurrent export never reads half a chunk.

6.5 Rendering, and Veusz command-line limitations

Tested with Veusz 3.6.2 and 4.2.1:

- `veusz --export=out.pdf file.vszh5` **cannot** open a `.vszh5`: the command line loads every file as a text script, gets an empty document, and the export fails. The embedded interface's `Load` command has the same limit.
- `--export-dpi` is not a Veusz option (exit code 2), and `%PAGE%` page templates are expanded only by the GUI's Export dialog, not the command line.
- Veusz 4.2.1 does not exit on `Quit()` in `--listen --quiet` mode (3.6.2 does), so `render_pages()` waits for its own end marker, gives Veusz two seconds, then ends it.
- Veusz 4.2.1 draws a scripted PDF at 72/150 of its size, in the top-left corner of each page. `AsyncExport.getDPI()` rounds the PDF resolution to the only one Qt's PDF engine reports (72) and lays the page out at it, but `ExportPDFRunnable` sets the printer to the requested `pdfdpi`, and `Export()` defaults to 150 [18], [22]. Veusz 3.6.2 did not round, so it is correct. `render_pages()` passes `pdfdpi=72`, which is right for both.
- What works: `veusz --listen --quiet` fed a script that calls `veusz.document.loader.loadHDF5Doc` and `Export(..., page=[i])`, which is what `render_pages()` does, or the Python package's `Document.load(path, mode='hdf5')`.

Fixing it in Veusz. All three limits come from short functions in Veusz itself [18], unchanged on its `master` branch as of 2026-10-11 [22]:

1. `CommandInterpreter.Load()` opens every file as UTF-8 text, and both the command line and the embedding API go through it.
2. The listen loop's `quitProgram()` only closes a window that `--quiet` never showed. Qt ends the application on `lastWindowClosed`, which is emitted only for the last *visible* window [29].
3. `ExportPDFRunnable` sets the printer to `pdfdpi` rather than to the resolution the page was laid out at.

The patch `upstream/veusz-4.2.1-vszh5-load-quit-pdfdpi.patch` makes `Load()` recognise an HDF5 file by its signature and open it with `mode='hdf5'`, makes `quitProgram()` also quit the application, and sets the PDF printer to the layout resolution. A ready-to-file issue text is in `upstream/VEUSZ_UPSTREAM_ISSUE.md`.

`upstream/check_veusz_patch.py` reproduces all three on any set of Veusz programs. "Text reaches" is the rightmost text on page 1 as a fraction of the page width; about 0.97 means the drawing fills the page.

Check	4.2.1	4.2.1 + patch	3.6.2
`--export .vszh5` to PDF	fails (exit −11, segmentation fault, no file)	OK, text reaches 0.97	fails (exit 0, no file)
`--export .vszh5` to PNG	fails (exit −11, no file)	OK	fails (exit 0, no file)
`Load() .vszh5` + `Export()` of 2 pages	fails (no PDF)	OK, 2 pages, text reaches 0.97	fails (no PDF)
`Quit()` in `--listen --quiet`	still running 30 s after `DONE` (killed)	exits 0.12 s after `DONE`	exits 0.11 s after `DONE`
Text `.vsz --export` to PDF (regression)	OK, but text reaches **0.47**	OK, text reaches 0.97	OK, text reaches 0.97
This package's `check_vszh5.py`	58 / 58	58 / 58	58 / 58

The patch is for submitting upstream [22]; this package does **not** need it. `render_pages()` uses the listen-mode route and `pdfdpi=72`, which work with patched and unpatched Veusz alike.

6.6 Statistics

Reported statistics include achieved rate, interval jitter, I/O latency percentiles, distinct commanded frequencies, power and temperature extrema, margin to the +75 °C internal limit [2], lock-loss and voltage-error tallies, and the overlapping Allan deviation of the reported carrier per IEEE Std 1139-2008 [9]:

$$\sigma_y^2(\tau) = \frac{1}{2\,(N-2m+1)\,\tau^2} \sum_{k=1}^{N-2m+1} \left(x_{k+2m} - 2x_{k+m} + x_k\right)^2, \qquad \tau = m\,\tau_0$$

with phase residuals $x_k = \tau_0 \sum_{i<k} y_i$ and fractional error $y_i = (f_i - f_c)/f_c$. Evaluating it from cumulative sums turns the inner double sum from $\mathcal{O}(Nm)$ into $\mathcal{O}(N)$ per averaging factor; the result was cross-checked against a direct summation and agrees to better than 10^{-9} relative.

> **Read this statistic correctly.** `FREQ?` returns the *commanded* setting, not a measurement of the output. The ADEV here characterises command fidelity and the logging time base — a non-zero value means commands were lost, retimed or applied late. It is **not** carrier phase noise, which is specified separately in [4] and cannot be inferred from this log. Measuring the carrier needs a counter or a phase-noise analyser.

6.7 PDF bookmarks and the LuaLaTeX session report

Bookmarks in the rendered PDF. `render_pages()` writes the outline with `vszh5_writer.add_pdf_bookmarks()` (pypdf [26]) after Veusz has rendered the pages, sets the title, subject and keywords, and opens the bookmark pane on load. The export script supplies this hierarchy:

```
QuickSyn session <session folder>
├── Overview (all quantities)
├── Time series
│   ├── Carrier frequency
│   ├── Output level
│   ├── Internal temperature
│   ├── Serial I/O latency
│   └── Sample interval
└── Stability
    └── Allan deviation of the reported carrier
```

Without pypdf the PDF is still written, without an outline, and the export says so.

The LuaLaTeX report (`quicksyn_session_report.tex`, compiled to `quicksyn_session_report.pdf`) is the LaTeX version of the session PDF. It contains:

- the instrument identity;
- the timing, signal and thermal tables;
- the stability statistics with their equations;
- every Veusz page, included as vector graphics straight from `quicksyn_session.pdf`;
- the session's files and IEEE-style references.

hyperref [27] turns the sections and subsections into the bookmark hierarchy (Contents; 1 Instrument … 6 Plots ▸ 6.1 Overview … 6.7 Allan deviation; 7 Files; 8 Provenance; References) and opens the bookmark pane. A simulated session is flagged in a red box on the first page.

Item	Behaviour
When	Whenever the plots are rendered (not with `--no-render`, `--no-latex`, or the controller's periodic refresh)
Engine	LuaLaTeX [28], run twice for the contents and bookmarks; `.aux`/`.out`/`.toc` removed, `.log` kept

Item	Behaviour
Finding `lualatex`	`--lualatex PATH`, then `QUICKSYN_LUALATEX`, then the `PATH`, then `C:\texlive\<year>\bin\windows` (TeX Live 2023 and later; `bin\win64` before), then MiKTeX's folder
Packages used	`fontspec`, `amsmath`, `booktabs`, `array`, `graphicx`, `xcolor`, `fancyhdr`, `hyperref`, `bookmark`: all in a standard TeX Live or MiKTeX installation
No TeX installed	The `.tex` is still written; compile it later with `lualatex quicksyn_session_report.tex` (twice)
Render failed	The report is still written, with a note in place of the plots
Failure	The first LaTeX error line is printed and the `.log` kept; the data files are never affected

The equations in the report are the ones in §6.6 and §5.1, set as numbered LaTeX equations.

---

7. Model capability table

Transcribed from [4]. Frequency and power entry ranges clamp to the detected model on connect.

Model	Coverage	Opt. 1 low end	Resolution	Output level	Power control (opt. 2)	Mod. inputs	SCPI documented
FSW-0010	0.5–10 GHz	0.1 GHz	1 mHz	+15 dBm	−25 … +15 dBm (0.01 dB nom.)	yes	yes
FSW-0020	0.5–20 GHz	0.2 GHz	1 mHz	+13 dBm	−10 … +13 dBm (0.1 dB nom.)	yes	yes
FSL-0010	0.65–10 GHz	—	1 mHz	+15 dBm min, fixed	none	no	no
FSL-0020	0.65–20 GHz	—	1 mHz	+10 dBm min, fixed	none	no	no
FSL-E020	0.65–20 GHz	—	1 mHz	+10 dBm min, fixed	none	no	no
FSL-2740	27–40 GHz	—	1 Hz	+18 dBm typ (+17 min, +24 max), fixed	none	no	no
FSL-5067	50–67 GHz	—	1 Hz	+17 dBm typ (+15 min, +22 max), fixed	none	no	no
FSL-7682	76–82 GHz	—	1 Hz	+11 dBm typ (+10 min, +14 max), fixed	none	no	no

Only the FSW models list option 2; the Lite models list option 8 only and the mmW models options 3 and 8 [4]. Up to 0.0.6 the mmW min/max output specification was put in the control-range column and offered in the power box; 0.0.7 corrects that (§3.3). The one FSL-0020 tested reports an option word with options 1–7 and 9, modulation included; the GUI follows the datasheet, not the option word (§3.3, §10.1).

[1] names only the FSW-0010 and FSW-0020. The other models are offered with the same commands but flagged **unverified** in the Connection tab, and an unrecognised model falls back to the union of all documented limits with a warning. Lock-recovery, blanking, reference routing and list/sweep commands are expected to be common across the family, but that is an inference.

---

8. Memory hygiene

Per [5]: the 2112-byte and 16 MB SRAM areas clear on a power cycle; the 8 MB flash has **no direct erase**, so the two user-writable areas can only be cleared by overwriting them with the same commands that wrote them.

Area	Procedure	Cost
Instrument states (2 slots)	`*RCL 0`, then `*SAV 1` and `*SAV 2`	~0.3 s
List point table (32767 locations)	Write safe frequency/power with `,F` into each location	300 ms each → **~2.7 h for all**

The Memory tab implements both, with an ETA and an abort. The cheaper course is prevention, which is why **RAM-only list download is this application's default**: it is ~3000× faster per point and the points clear on the next power cycle.

---

9. Documented conflicts and how they are handled

#	Conflict	Handling
1	Post-reset blanking: `ON` in [1, Table 2a] vs `off` in [2, Table 1]	Never assumed; `STAT?` is read back after every reset and the observed value reported
2	SCPI `*IDN?` field order undefined; the [1] example puts `0` in the version position, below the firmware-100 gate it requires	**Resolved on hardware (0.0.8):** manufacturer, model, option word, serial, firmware — NI's panels show the FSL-0020 as serial 1530200407, firmware 0008, options 017F. 0.0.7 had serial and firmware swapped. Raw fields still shown verbatim
3	Query double-issue required on SPI, temperature-only on native, unstated for SCPI	`TEMPERATURE_DOUBLE_QUERY`, default `True`, recorded in the session metadata
4	Response terminator unspecified	Reader accepts CR, LF or CRLF, times out cleanly
5	0.01 dB resolution quoted in [4] vs tenth-dBm command units in [1]	Quantised to 0.1 dB; the quantised value is logged

#	Conflict	Handling
6	mlHz default vs GHz-suffixed examples in compound commands	Shortest exact spelling, pre-flighted against the buffer (§4)
7	`SWE:FAST:FREQ:SETUP` example in [1] has a missing comma	Read as the eight documented fields plus the optional `RUN`
8	"Start NF Sweep" example printed as `SWE:NORM:FREQ:SETUP 1`	Treated as a typo for `SWE:NORM:FREQ:STAR 1`
9	32767 addressable list points vs "32,000 points" in [4]	Command-set limit used for range checks, datasheet figure surfaced in the UI
10	`DIAG:BAUD` assumed to apply to USB	Documented as debug-port only; USB fixed at 115200
11	`POW +/- XX.X` in [1, Table 5] with only a negative example (`POW -8.3`); whether a `+` is required, accepted or rejected is not stated	Sign always written (`POW +12.0`); the native command is the fallback, chosen automatically when `POW?` shows no change (§3.3)
12	[1] marks native Set Output Power "*When option is present" but says nothing of the kind for SCPI* `POW`; IDN? does not report the options	Power is assumed to need option 2 in both forms (as [4] states); the level is read back after every change and a mismatch names option 2
13	Native Get Power (`0D`) reply format for negative levels not stated in [1, Table 4]	Two's complement **confirmed on hardware**: `FFE2` = −3.0, `FF9C` = −10.0 dBm
14	[1] says SCPI needs firmware 100 or higher	The FSL-0020 reports firmware 0008 and answers SCPI; the rule is shown as information, no dialog
15	`STAT?` example `00A8` in [1, Table 5]	The hardware replies `0xA1` (and `DIAG:MOD? 0x0`); both forms are parsed, the simulator now uses the hardware form
16	Option word lists option 2 on an FSL-0020; [4] lists only option 8 for the Lite	The datasheet decides the GUI's gating; a power meter agreed (§3.3)
17	`OUTP:ROSC:STAT` in [1, Table 5]; on the FSL-0020 it did not change `STAT?` bit 5, while REF OUT switches with NI's panel (native commands)	**Settled 2026-10-14:** `OUTP:ROSC:STAT` does switch REF OUT; bit 5 is stale on that unit. `OUTP:ROSC:STAT?` trusted when it answers, otherwise the state is shown as not reported (§3.5)
18	[3] describes writing power and reference calibration into the unit (*Write Cal to UUT*, *Store in Flash*, *Write Ref Cal to UUT*); [1] publishes only `DIAG:CAL:REF:DAC`	Only the documented command is sent; power tables are kept in this program (§3.7)
19	[4] gives 0.65 GHz as the FSL-0020's lower limit; the unit, asked for 100 MHz (by NI's panel or by `FREQ 100000000000`), went to **656.062 MHz**; NI's panel displays 100 MHz regardless	The 0.65 GHz limit is kept; requests below it are refused here. Whether 0.650 GHz itself is reached or also clamps near 656 MHz is open (§13)

10. Verification status

Validated headlessly against the built-in simulator, with Qt on the offscreen platform, and once against hardware: the 0.0.7 controller on an FSL-0020 (§10.1), whose findings 0.0.8 fixes. **The 0.0.8 fixes themselves have not yet been run on the instrument.** The first table was run for **this build** (0.0.8, 2026-10-12) on Python 3.12.3 (the declared target) and Python 3.13.16, PySide6 6.12.0. The environment had:

- real Veusz **3.6.2** (Ubuntu 24.04 package, PyQt5 / Qt 5.15.13);
- real Veusz **4.2.1** (the version conda-forge ships), built here from the PyPI source against PyQt6 6.6.1 / Qt 6.4.2;
- the same 4.2.1 with the upstream patch of §6.5;
- **LuaLaTeX** from TeX Live 2023 (LuaHBTeX 1.17.0, hyperref 7.01h).

The 3.12 run is the one that counts: it found a defect the 3.13 runs had hidden (0.0.3 history entry). The second table is the 0.0.6 build, whose code 0.0.7 does not change outside the GUI's power, RF-off and console code; the rows that `check_vszh5.py` and `check_console.py` cover were re-run in this build (first table). The third table lists older results. Rows marked † are measurements from 0.0.4/0.0.5 that were not repeated. Counts vary by a few samples with timing.

Check (0.0.10 build, 2026-10-14)	Result
`tests/run_all.py`, Python 3.12.3 with Veusz 4.2.1, and Python 3.13.16	6 / 6 checks pass on both, also from the extracted archive
Path loss	constant; CSV with GHz, MHz and unit-less Hz columns and S_{21} given as negative dB; Touchstone v1 `DB`/`MA`/`RI` of a synthetic cable ($0.6\sqrt{f} + 0.05f$ dB) recovered to 10^{-5} dB; v2 with `21_12` and `12_21`, noise data ignored; `.s1p` refused; no extrapolation
Power tables	interpolation and band edges; level correction against a synthetic FSW (error $-(0.5 + 0.15f) + 0.02P_s$ dB) within 0.05 dB of the exact solution, clamped at +13 dBm, `None` outside the band; JSON and CSV round trip; another file kind and a newer schema refused
OCXO tuning	from DAC 30000, 33500 and 12000 against a slightly curved model (0.3 Hz/count at 10 GHz): within ±20 Hz in ≤ 6 measurements; DAC clamped at 65535
PyVISA reader (fake E4418B and CNT-90)	`*IDN?`, `SENS:FREQ 12500000000` then `MEAS?`, `-3.21000E+00` → −3.21 dBm; resource-not-found and `OVLD` reported as errors; the install hint when PyVISA is missing
GUI, simulated FSL-0020	20 points: 3 typed (Enter records and sets the next point), 17 by the fake meter in an automatic run; saved and reloaded on reconnect; Main tab "+7.80 dBm expected at 10.500000 GHz", out-of-band noted; CSV and `.vszh5` export
GUI, OCXO	DAC 30000 → 31240 in 3 counter readings (+1.8 Hz at 10 GHz); restore; saved value re-sent on connect
GUI, simulated FSW-0020	15 points (5 frequencies × 3 levels) by the fake meter; 0 dBm at RF OUT commanded as `POW +2.0` at 10 GHz and re-corrected to `POW +3.1` on moving to 18 GHz; another unit's table not extended

Check (0.0.10 build, 2026-10-14)	Result
10 MHz output, bit 5 stuck (no query)	`OUTP:ROSC:STAT OFF`, `STAT?`, `0800`, `STAT?`, one warning; then `OUTP:ROSC:STAT ON`, `STAT?` only; LED *not reported (last sent OFF)*; *Read all* leaves the checkbox alone
10 MHz output, bit 5 stuck, query answers	`OUTP:ROSC:STAT OFF`, `OUTP:ROSC:STAT?`, `STAT?`; "confirmed by OUTP:ROSC:STAT?"; no native command
NI panel vectors (§10.2)	the 11 power words, `0F`/`08`/`06` switches and the 100 MHz frequency word reproduced
Example session and calibration (`examples/`)	41 s simulated session (414 samples), 7-page bookmarked PDF and 9-page LuaLaTeX report; simulated calibration rendered by Veusz 4.2.1

Check (0.0.9 build, 2026-10-12)	Result
`tests/run_all.py`, Python 3.12.3 with Veusz 4.2.1, and Python 3.13.16	5 / 5 checks pass on both, also from the extracted archive
10 MHz output, SCPI ignored	`OUTP:ROSC:STAT OFF`, then native `0800`; output follows; next switch sends `0801` first; status bar says the native command is kept
10 MHz output, both ignored	warning asks for a power-meter check; the native command is **not** kept
`OUTP:ROSC:STAT?` unanswered	one attempt, then skipped for the session; the switch is confirmed by `STAT?` alone; *Read all settings* completes without error and without re-sending the query

Check (0.0.8 build, 2026-10-12)	Result
`tests/run_all.py`, Python 3.12.3 with Veusz 4.2.1, and Python 3.13.16	5 / 5 checks pass on both, also from the extracted archive
IDN? of the FSL-0020	parsed as model FSL-0020, option word 0000017f → options 1–7, 9, serial 1530200407, firmware 0008
Enum arguments as combo boxes return them	`set_reference_source("EXT")`, `set_fm_mode(3)`, `setup_list(…, 2, 0)` all send the right command; the reference combo, FM-mode combo, *Configure sweep* and *Set up and run list* driven through the GUI without error
Typing (§3.6)	0.8 GHz typed five different ways → `FREQ 800000000000`; 13 decimals rounded to 1 mHz; 0.1 GHz refused by Enter and by *Set*, nothing sent, message shown
Reference output	confirmed (SCPI); SCPI ignored → native `0800` used and confirmed; ignored by both → warning; checkbox follows `STAT?`
Reference source and LEDs	EXT with signal → *External … IN USE*; signal removed → *NO SIGNAL at REF IN* and a warning after the 1.5 s re-check; back to INT → *Internal … IN USE*
Auto-read	1 s interval: ≥ 2 full reads in 2.4 s, typing in the frequency box not overwritten, stops when unticked

Check (0.0.8 build, 2026-10-12)	Result
Temperature flag	27.0 → 54.2 °C flagged *suspect* with a warning; 54.2 → 54.3 not
Timing	no `time.monotonic` call left (AST check); simulated latencies 1.6 ms (0.0.7 on Windows/3.12 logged 0 or 16 ms)
The hardware session re-exported with the 0.0.8 export script	*IDN?* table now reads serial 1530200407, firmware 0008, options 1–7, 9 (the 0.0.7 summary showed the serial as the software version)
Example session	41 s simulated, 0.0.8; bookmarked plots PDF and LuaLaTeX report built with Veusz 4.2.1

Check (0.0.7 build, 2026-10-12)	Result
`tests/run_all.py`, Python 3.12.3, `QUICKSYN_TEST_VEUSZ_EXE` = Veusz 4.2.1	5 / 5 checks pass (release metadata, smoke, reduction, `.vszh5`, console)
`tests/run_all.py`, Python 3.13.16	5 / 5 checks pass; `check_vszh5.py` 53 passed, 5 skipped (the in-process Veusz checks: no Veusz package for 3.13 here)
`check_release_metadata.py`	148 / 148: every file at 0.0.7 with a 0.0.7 history entry, SPDX tags, `LICENSE` SHA-256, `--version`, About dialog, README PDF bookmarked and current
`check_vszh5.py`, Python 3.12.3 with Veusz 4.2.1	**58 / 58** (writer, export, render plumbing, page scale, bookmarks, re-render, LuaLaTeX report)
`check_console.py`	all pass with the new *Send* line in place
Native Set Output Power	`030078` (+12), `03FFE2` (−3) — both [1, Table 2a] examples — and `030082` (+13), `03FF9C` (−10)
SCPI form	`POW +12.0`; the vendor example `POW -8.3` still byte-exact
Read-back	`POW?` after every *Set power*, `FREQ?` after every *Set frequency*; 0.05 dB tolerance
Auto fallback (simulated firmware that ignores SCPI `POW`)	sends `POW +12.0`, `POW?`, `030078`, `POW?`; selector switches to *Native*; next level sent as `03FFE2`
Unit without option 2 (simulated)	level stays at +12.0 dBm; read-back red, status bar names option 2, console `NOTE !` line
FSW-0020 limits	−10.0 and +13.0 dBm accepted; −10.1 and +13.1 refused; box range −10 … +13 dBm
Fixed-output models (FSL-0020, FSL-E020, FSL-2740)	driver refuses `POW` and power sweeps; GUI disables the power box, *Set power* and both power-sweep types and shows why; FSW-0020 re-enables them
Enter in the boxes	typing `7.25` + Enter sends `FREQ …`, `FREQ?`; `-5.0` + Enter sends `POW -5.0`, `POW?`
Read (0D)	sends `0D`, decodes `FFE2` as −3.0 dBm
Console *Send* line	`*IDN?`, `0D`, `POW?` sent as queries and `030078` as a write by Enter; a forced query on a write reported, not raised; a 71-byte command refused by the 64-byte check
RF OUTPUT OFF	button (Main tab) and toolbar (from the Monitor tab) each send `OUTP:STAT OFF`, `STAT?`; status bar "confirmed"; checkbox cleared; stays enabled on fixed-output models

Check (0.0.7 build, 2026-10-12)	Result
Appearance	Main tab and console inspected in light and dark themes; RF OFF buttons drawn #c62828 in both (0.0.7 draft drew the Main-tab button pale red under the platform bevel; fixed with a border rule)
Example session (`examples/`)	41 s simulated session recorded with 0.0.7 at 10 Hz (411 samples, levels set with read-back, RF OFF at the end); export with Veusz 4.2.1: 7-page PDF with 10 bookmarks, 9-page LuaLaTeX report with 17 bookmarks
Archive	see §11; checked with 7-Zip 16.02 `t`, extracted to a clean folder and `run_all.py` re-run there

Check (0.0.6 build, 2026-10-11)	Result
`tests/run_all.py`, Python 3.12.3, `QUICKSYN_TEST_VEUSZ_EXE` = Veusz 4.2.1	5 / 5 checks pass (release metadata, smoke, reduction, `.vszh5`, console)
`tests/run_all.py`, Python 3.13.16	5 / 5 checks pass; `check_vszh5.py` skips the 5 in-process Veusz checks (no Veusz package for 3.13 here) and runs the program checks through Veusz 4.2.1
`check_release_metadata.py`	148 / 148: every file at 0.0.6 with a 0.0.6 history entry, SPDX tags in every Python file, `LICENSE` SHA-256, `--version`, About dialog and Help ▸ About, README PDF bookmarked and current
`check_vszh5.py` with real Veusz	**58 / 58** each with 4.2.1 (in-process and program), patched 4.2.1, and 3.6.2 (in-process and program)
PDF page scale (0.0.6 fix)	rightmost text at 0.97 of the page width on every page (0.89 on the log–log ADEV page) with 4.2.1, patched 4.2.1 and 3.6.2, program and in-process; the 0.0.5 writer with 4.2.1 gave 0.46–0.48, i.e. the drawing in the top-left corner
PDF bookmarks	10 bookmarks in 3 levels exactly as in §6.7; title and subject set; page mode `/UseOutlines`; a bookmark pointing outside the PDF is refused
Re-render into the same folder	returns the same 8 files (0.0.5 returned none)
LuaLaTeX report	compiles in 2.2 s (two runs) with **no LaTeX warnings**; 9–10 pages; bookmarks: Contents, 8 numbered sections, 7 plot subsections under 6 Plots, References; outline pane opens; `.aux`/`.out`/`.toc` removed; no unescaped underscore in the text; simulated session flagged; pages inspected as images
Export CLI with render	writes `quicksyn_session.pdf`, `quicksyn_session_report.tex` and `.pdf`; prints the report line the controller shows in its status bar
Example session (`examples/`)	40 s simulated session driven through the GUI at 10 Hz (401 samples, four commanded steps); export with Veusz 4.2.1: 7-page bookmarked PDF, 9-page LuaLaTeX report with 17 bookmarks (Contents, 8 sections, 7 plot subsections, References); pages inspected as images
README PDF	built with `tools/build_readme_pdf.py` (pandoc 3.1.3, LuaLaTeX): 29 pages, no missing glyphs, bookmarks for all 14 sections and their subsections

Check (0.0.6 build, 2026-10-11)	Result
`.vszh5` written without Veusz	in an interpreter where `veusz` cannot be imported, `--no-render` writes the full document; `python -X importtime` lists no Veusz module
`check_console.py`	all pass on 3.12.3 and 3.13.16: write/query/error/empty-reply/CR-LF lines; reply arrow under the command arrow (column 18) for GUI and POLL; 5000 exchanges from a worker thread all filed on the GUI thread; a simulated session's 142 exchanges = 142 rows of `transactions.csv`; 200 200 exchanges while paused hold 5.3 MB, Resume 12–106 ms, view ≤ 5000 lines with the … notice, all 200 own commands kept; filter debounced, Enter immediate, paused filter searches the frozen view
SCPI console, 0.0.5 vs 0.0.6 (same probe)	0.0.5: 45.5 MB held, 62 s frozen on Resume, 1.6 s per filter keystroke, GUI reply column one off; 0.0.6: figures in §5.4
`check_smoke.py`	commands, interlocks, sampler, ring spills, GUI session logging, the controller's `.vszh5` export mid-session and at stop (= `telemetry.csv`), capability gating
Upstream Veusz patch	`patch -p1 --dry-run` clean on the 4.2.1 source; results of `upstream/check_veusz_patch.py` in §6.5
Recovered `vszh5_writer.py` 0.0.5	rebuilt byte-for-byte from 0.0.4 plus the 0.0.5 changes kept in the project notes: SHA-256 `2cdf43d2…ae33c1c`, as recorded for 0.0.5
Archive	see §11; checked with 7-Zip 16.02 `t`, extracted to a clean folder and `run_all.py` re-run there

Check (unchanged since 0.0.4/0.0.5)	Result
`STAT? → 00A8` and `DIAG:MOD? → 10` decoding	matches [1, Table 5] bit maps, lock-sense inversion included
Unit conversions, power and dwell quantisation	pass
34 formatted commands vs the vendor worked examples	pass; longest 58 bytes of 64
`LIST:PVEC` against both [1, Table 2b] examples	byte-exact
Over-long compound command	refused with rounding advice
FM interlock on lists and sweeps	both blocked
8 out-of-range parameters	all rejected
Sampler at 20 Hz	40 samples in 2.0 s, 0 errors
Ring overflow, 250 records into capacity 100	spills of 100/100/50, flat RAM
GUI construction, 7 tabs, 3 themes	pass
End-to-end GUI session	3 commanded frequencies + power + RF state logged to `state_events.csv`, 11 telemetry rows, 62 transactions
Capability gating (FSL-2740)	Modulation tab hidden, entry clamped to 40 GHz
ADEV vs direct $\mathcal{O}(Nm)$ summation	agrees to $< 10^{-9}$ relative over 4 averaging factors
Moving statistics vs `np.convolve`	agrees to 10^{-9}
Dataset types in real Veusz	`Dataset` (1d, with `serr`/`perr`/`nerr`), `DatasetDateTime`, `DatasetText`; tags restored
Data precision †	288 000-sample session bit-exact after loading in Veusz; carriers 1 mHz apart stay distinct; commanded values equal reported ones

Check (unchanged since 0.0.4/0.0.5)	Result
Escaping	names with `/` and `` ` `` restored; `vz_text` identical to Veusz `latexEscape`; `Run_01 ^ {x} [y] C:\data` renders literally
Local time across DST	offsets −4 h → −5 h across the 2026-11-01 change (America/New_York)
Pages	overview + 6 single pages, every trace pointing at an existing dataset, for local, UTC and elapsed axes
8-hour session, 10 Hz †	288 000 samples reduced, written (6.2 MB `.vszh5`) and rendered to PDF + 7 PNGs in ≈ 27 s
Controller export while logging	`.vszh5` rewritten mid-session (11 samples) without stopping acquisition; at stop it holds every sample (= `telemetry.csv`)
No text `.vsz`, no leftover `.part`	checked in the command-line run and the GUI session
Telemetry writer shutdown on Python 3.12	thread exits cleanly after `stop()` (regression for the `_stop` shadowing defect)
3.13-only API	no use of `os.process_cpu_count` (added in 3.13 [16]); `available_cpus()` uses scheduler affinity with an `os.cpu_count()` fallback

10.1 First hardware session (FSL-0020, 2026-10-12)

NI/Phase Matrix FSL-0020, serial 1530200407, firmware 0008, option word 017F; COM5 over USB; the 0.0.7 controller on Python 3.12.0 (Windows), Qt.py not installed (the PySide6 fallback worked); 600 s logged at 2 Hz, 5194 exchanges, 0 transport errors; a power meter on RF OUT.

#	Observed	Cause	0.0.8
1	Choosing External reference: “ ‘str’ object has no attribute ‘value’ ” (three times)	PySide6 returns `StrEnum` combo data as `str`	Driver converts enum arguments (§3.5)
2	0.800 GHz could not be typed; the arrows worked	Stock spin-box validation (§3.6)	`EntrySpinBox`
3	10 MHz output switch “does not work” after *Read all*	`OUTP:ROSC:STAT OFF` sent three times; `STAT?` kept bit 5 set; never read back. REF OUT **does** switch with NI’s panel (native commands) on a power meter	0.0.8: read-back and native fallback; 0.0.9: native kept once it works, `OUTP:ROSC:STAT?` optional (§3.5)
4	Level unchanged on the power meter, also with NI’s panel; changes with frequency	No working power control on this FSL-0020 (§3.3)	None needed: 0.0.7 already disabled it; documented
5	`POW?` unchanged by RF on/off	`POW?` is the stored setting	Read-back says “(the level setting; the RF output is OFF)”
6	Every exchange 0.0 ms in the console; latencies only 0, 15 or 16 ms; intervals 312–688 ms around 500 ms	Python 3.12 on Windows: `time.monotonic()` ticks at 15.6 ms (GetTickCount64); 3.13 moved it to QueryPerformanceCounter [33]	`time.perf_counter()` throughout (§10)
7	Temperature 27.0 → 54.2 °C right after `POW 5.0`, back to 27.6 °C after `OUTP:STAT OFF`; again 28.4 → 57.0 °C after `POW -10`	Reporting artefact: the ratio is 2.00 each time and a case cannot step 27 °C in 2.5 s	Steps ≥ 10 °C within 60 s flagged *suspect*; the logged values are left as reported

#	Observed	Cause	0.0.8
8	*IDN?* serial shown as “software version 1530200407”	Fields 4 and 5 swapped (§9 #2)	Fixed in GUI and export
9	`STAT? 0xA1`: external reference *detected* while internal in use	Bit 0 is presence at REF IN, not the source in use	Two new LEDs (§3.5)
10	`0D` replies `0032`, `0000`, `FFE2`, `FF9C`	Two’s complement confirmed	§9 #13

Reduction of that session (export 0.0.8): 1201 samples, 2.000 Hz achieved, latency mean 11.7 ms / p95 16 ms / max 188 ms (quantised, item 6), temperature 26.8–57.5 °C (item 7), frequency fixed at 1 GHz, so the ADEV is zero at every τ and has no log-scale plot.

Not yet verified: the 0.0.8, 0.0.9 and 0.0.10 changes on the instrument (the REF OUT behaviour of 0.0.10 follows the 2026-10-14 observation, §10.2, but has not itself been run there); the Calibration tab with a real meter or counter (only fakes here) and PyVISA on Windows; output power on an FSW with option 2; whether the FSL-0020’s REF OUT follows the native `08xx` sent by this program (it does with NI’s panel) and whether `STAT?` bit 5 then changes; the LuaLaTeX report with TeX Live 2025 or MiKTeX on Windows (verified with TeX Live 2023 on Linux; the packages it uses are standard in both); anything requiring hardware — real round-trip latencies, the actual response terminator, whether the SCPI temperature query needs its second issue and the `*IDN?` field order. The Veusz render is verified on Linux with Veusz 3.6.2 and 4.2.1, **not yet on Windows** in your conda-forge environment; in particular, whether the Windows `veusz.exe` (a windowed program) reads a script piped to its standard input in `--listen` mode is untested. If it does not, `method="auto"` falls back to the Python package, which conda-forge installs. Also not run: an actual `mamba env create` (the container has no conda-forge access). The presence of `qt.py` 2.0.3 and `veusz` 4.2.1, `h5py` 3.16.0 (win-64 included) and `pypdf` 6.20.0 (noarch) on conda-forge were confirmed from their package pages [17], [19], [26], which is weaker than a successful solve.

10.2 Second hardware check (FSL-0020, 2026-10-13/14)

Reported by the user from the same FSL-0020 (serial 1530200407), with NI’s soft front panel (SFP release V1.0.21), the 0.0.9 controller, a power meter and photographs of the unit.

#	Observed	Conclusion	0.0.10
1	`OUTP:ROSC:STAT OFF` / `ON` from this program switch REF OUT off and on (power meter); `STAT?` bit 5 stays set	SCPI works; the status bit is stale	§3.5
2	NI’s panel log: REF OUT `08 00/08 01`, reference source `06 01/06 00`, RF `0F 00/0F 01`	The native commands this program sends are the panel’s	Regression vectors
3	NI’s power wheel: `03 FF EA` … `03 FF C5`, `03 00 00`, `03 00 96`, `03 00 C8`, `03 FF 7E`	−2.2 … −5.9, 0, +15, +20, −13 dBm: byte-identical to `native_power_command()`. The power meter did not follow the panel either, so the fixed level is the unit, not the command	Regression vectors
4	Panel *Set Frequency* `0C 00 17 48 76 E8 00` for 100 MHz	`0x174876E800` = 10^{11} mHz = 100 MHz	Regression vector
5	100 MHz asked (panel, or `FREQ 100000000000` here) → output at **656.062 MHz**; the panel still displays 100 MHz	The firmware clamps below-range requests; NI’s panel does no range check (it also sent +20 dBm)	§9 #19; limit kept at 0.65 GHz

#	Observed	Conclusion	0.0.10
6	Output **+5.3 dBm at 10 GHz**, **−10 dBm at 20 GHz** on the power meter, the same from NI's panel and from this program	Below the +10 dBm datasheet minimum [4] by 4.7 dB and 20 dB; see below	§3.7 to measure the unit's own curve
7	Photos: label "QuickSyn Lite Synthesizer", MODEL FSL-0020, CCN 15302, S/N 15302-00407; connectors REF IN, REF OUT, USB, POWER/SPI, RF OUT only	Lite confirmed; no PULSE/AM/FM inputs, as [4] says	—
8	NI's calibration panel needs GPIB and an E4418B-class meter	Program-side calibration instead	§3.7

Can cable loss explain item 6? Coaxial loss in dB is the sum of a conductor term growing as $\sqrt{f}$ and a dielectric term growing as f [36]:

$$L(f) = a\sqrt{f} + b\,f, \quad a, b \geq 0 \quad \Longrightarrow \quad \sqrt{2} \leq \frac{L(20\ \text{GHz})}{L(10\ \text{GHz})} \leq 2.$$

If the unit put out its minimum +10 dBm at both frequencies, the path would have to lose $10 - 5.3 = 4.7$ dB at 10 GHz and $10 - (-10) = 20$ dB at 20 GHz, a ratio of $20/4.7 = 4.3$, outside $[1.41, 2]$. So a cable alone cannot account for both readings; a sensor out of its range or without its cal factor, a damaged connector or adapter (which can resonate), or a real output shortfall remain. Measuring the path's S_{21} (or a substitution with a known source) and loading it as a `.s2p` in the Calibration tab separates the two (§13).

---

11. Revision history

Revision	Date	Change
0.0.1	2026-10-09	First internal draft. Transport, driver, capability table, simulator, ring buffer, sampler, writer thread, seven-tab GUI, theming, session logging.
0.0.2	2026-10-09	Line-by-line verification against [1]–[5]. Compound-command frequency encoding rewritten after the smoke test found every sweep setup overflowing the 64-byte buffer; dwell quantised to 5 µs; `*IDN?` relabelled with all raw fields shown and the firmware gate made advisory; `DIAG:BAUD` corrected to debug-port only; Modulation tab gated on model capability; ring-buffer floor documented; references corrected and extended; conflict list grown from 6 to 10 entries with exact citations; companion reduction and Veusz export script added.

Revision	Date	Change
0.0.3	2026-10-09	Release metadata. Author, company, email and the GPL-3.0-or-later notice added to every header, with SPDX tags, module dunders, a `--version` flag and an About dialog (version, author, company, `mailto:` link, licence, GPL notice, runtime versions); `LICENSE` and `tests/` added; template placeholders removed. **Fixed (Python 3.12 only):** `TelemetryWriter` kept its stop flag in `self._stop`, shadowing `threading.Thread._stop()`, so `join()` raised `TypeError: 'Event' object is not callable` when logging stopped — the 0.0.2 controller has this defect on 3.12. **Fixed (Python 3.12 only):** the export script defaulted its worker count with `os.process_cpu_count()`, which exists only from 3.13 [16]; it now uses `available_cpus()`. Corrected an inaccurate “PEP 695” remark in the interpreter-guard comment.

Revision	Date	Change
0.0.4	2026-10-11	**Veusz output is a `.vszh5` written with the author's method from `isp_speed_monitor.py` 1.3.0**, now the standalone module `vszh5_writer.py` (`VeuszWriter`, `VszDoc`, `vz_text`, `epoch_to_veusz`, `find_veusz_exe`, `render_pages`); no text `.vsz` (which kept 7 significant digits: up to 500 Hz error below 10 GHz, 5 kHz above). `export_veusz()` follows the speed monitor's layout: tagged `raw/`, `events/`, `stability/`, `meta/` datasets, UTC and local date axes, overview + per-quantity pages, commanded values overlaid. **The controller writes the `.vszh5` itself** at stop, on demand (Ctrl+E) and every N minutes while logging, via the export script in a separate process; Open in Veusz; atomic chunk files. Rendering drives the Veusz program in `--listen` mode (the command line cannot open `.vszh5`). Fixes found on the way: titles with _ printed as subscripts (0.0.3); commanded/reported GHz differed in the last bit; Veusz 4.2.1 listen mode does not exit on `Quit()`. Checked against real Veusz 3.6.2 and 4.2.1.

Revision	Date	Change
0.0.5	2026-10-11	*Not released as an archive; these changes ship in 0.0.6.* **Live SCPI console** (View ▸ SCPI console, Ctrl+Shift+C): every command and reply as it happens, with time, source (GUI or telemetry poll), latency and errors; colour-coded for light and dark themes; polls hidden by default; filter, pause, copy, save; thread-safe through a queued signal, batched rendering, bounded memory (§5.4). Tab pages scroll, so the window no longer has to be taller than the tallest page, and it opens at 1180 × 960 px clamped to the screen. **Render fixes** in `vszh5_writer.py`: the PNG path is bound outside the `exec()` string (a Windows path was unescaped twice); `method="auto"` falls back to the Python package when the Veusz program fails, not only when it is missing; a program that exits before reading its script or cannot start is reported, not raised as `BrokenPipeError`/`OSError`; the output pipe is closed and Veusz never left running. New tests for all four. **Veusz upstream patch** and issue text for the two Veusz limits (§6.5). **Documentation:** the licence question on the `isp_speed_monitor.py` code is closed (§13). History entries put in ascending order in every file.

Revision	Date	Change
0.0.6	2026-10-11	**First package since 0.0.4.** 0.0.5 was never released as an archive, so its changes ship here as well. **Live SCPI console bounded while paused:** 0.0.5 queued every exchange during a pause and drew them all on Resume. 200 000 exchanges (≈ 8 h at 2 Hz) held 45.5 MB and, with polls shown, froze the window for 62 s, during which even *RF output off (panic)* had to wait. Now nothing is queued: GUI and POLL keep separate stores of 20 000, so polls cannot push your own commands out of the searchable history. A redraw draws at most the 5000 lines the view holds, with a ⋯ notice for the rest. Lines are inserted with a text cursor (≈ 10× faster than `appendHtml`) and line up exactly. The filter waits for a 250 ms typing pause (Enter applies at once) and, while paused, searches the frozen view. CR/LF in a reply shows as ↵; an empty reply is labelled (§5.4). **PDF bookmark hierarchy** in the rendered session PDF (`add_pdf_bookmarks()`, pypdf). **LuaLaTeX session report**, the LaTeX version of the session PDF: statistics, equations, every plot page as vector graphics, references, with hyperref bookmarks (§6.7). New options `--no-latex` and `--lualatex`. **Fixed (Veusz 4.x):** PDF pages were drawn at 48 % scale in the top-left corner, because Veusz 4.2.1 lays a PDF out at 72 dpi but sets the printer to `Export()`'s default `pdfdpi=150`; PDFs are now exported with `pdfdpi=72`. The PNGs were always right, which is why the 0.0.4 checks, which inspected PNGs, missed it. **Fixed:** a second render into the same folder returned no files, so the controller showed no plots. **Veusz upstream patch** now fixes three limits (`.vszh5` in `--export`/`Load()`, `Quit()` in listen mode, the PDF scale), with a reproduction script (§6.5). **Tests:** all re-targeted to 0.0.6; new `check_console.py`; `check_vszh5.py` grows from 36 to 58 checks (render plumbing,

Revision	Date	Change
0.0.7	2026-10-12	**Output power.** Reported: the level could not be changed. Fixed: Enter now applies the frequency and power boxes (0.0.6 applied only on the Set buttons); every change is read back with `POW?` / `FREQ?` and a mismatch is shown in red, in the status bar, the session log and the console, naming option 2; the level is written with its sign (`POW +12.0`); a *Command* selector adds the native Set Output Power command (`030078`, [1, Table 2a] examples reproduced) with automatic fallback; fixed-output models (FSL Lite, FSL mmW) no longer offer a level box or power sweeps, and the mmW min/max output specification is no longer offered as a control range. **Bring-up aids:** *Read (0D)* (native Get Power) and a console *Send* line. **RF OUTPUT OFF:** red button on the Main tab and on a *Safety* toolbar, confirmed with `STAT?` (§3.4). **Documentation:** §2.1 pyserial vs PyVISA; §3.3, §3.4; conflicts 11–13; model table corrected. **Tests:** `check_smoke.py` covers all of the above.

Revision	Date	Change
0.0.8	2026-10-12	**First hardware session** (FSL-0020, §10.1). **Fixed:** reference-source crash (PySide6 returns `StrEnum` combo data as `str`; the driver now converts all enum arguments); frequencies below 1 GHz could not be typed and the old value was re-sent (new `EntrySpinBox` for every numeric box, out-of-range typing refused with a message); *IDN?* serial and firmware swapped (order confirmed on hardware, option word decoded, no firmware dialog); 15.6 ms timing on Windows/Python 3.12 (`perf_counter`); 10 MHz output switch unchecked (read-back of `OUTP:ROSC:STAT?` and `STAT?`, native `08xx` fallback; reference source likewise with `06xx`). **Added:** Internal/External 10 MHz reference LEDs; *Auto-read all settings every N s*; "RF output is OFF" in power read-backs; suspect-temperature flag. **Export:** *IDN?* labels from the raw fields, so 0.0.7 sessions are labelled correctly. **Simulator:** hardware reply forms (`0xA1`), REF IN presence, native switches. **Documentation:** §3.3 hardware result and the calibration-panel checks, §3.5, §3.6, §5.5, §10.1, conflicts 2 and 13–16.
0.0.9	2026-10-12	**10 MHz reference output.** The user confirmed that the FSL-0020's REF OUT switches off (NI's panel, power meter), so the fault is the SCPI path, not the hardware. `OUTP:ROSC:STAT?`, never seen to answer on that unit, is now optional: in 0.0.8 a time-out there would have stopped the switch before the native fallback and failed every *Read all settings* and auto-read. The native `08xx` command is kept for the session once it has worked where the SCPI form did not. The warning no longer suggests the output cannot be switched. README §3.5, §9 #17, §10, §10.1, §13 corrected.

Revision	Date	Change
0.0.10	2026-10-14	**Calibration tab** (§3.7), in place of NI's GPIB-only calibration panel: power against frequency (and against level on power-control models) from typed readings or any SCPI power meter through PyVISA, with path loss from a constant, CSV or Touchstone `.s2p`; automatic runs; table, plot against the datasheet, JSON/CSV/`.vszh5`; a per-unit active table giving the expected level at RF OUT on the Main tab and, on power-control models, a corrected commanded level. OCXO tuning with `DIAG:CAL:REF:DAC` against a counter (secant search, ±20 Hz at 10 GHz), restore, and a saved value re-sent on connect. **Nothing undocumented is written to the unit. 10 MHz output:** SCPI works on the FSL-0020 and bit 5 is stale (§10.2), so `OUTP:ROSC:STAT?` is trusted when it answers and the state is otherwise shown as not reported, without false warnings or a native fallback. Read-backs cleared on connect. `set_reference_dac` docstring cites [3]. **Tests:** new `check_calibration.py` (with the NI panel vectors). **Examples** regenerated, plus a simulated calibration. **Documentation:** §3.5, §3.7, §9 #17–19, §10, §10.2, §12 [35]–[37], §13, §14.1. pyvisa and pyvisa-py added to the environment (optional).

---

12. References

#	Reference	URL
[1]	National Instruments, “QuickSyn Microwave Synthesizers: Communications Specifications,” Doc. No. 5580510-01, Santa Clara, CA, USA, 2014. (Revision not printed.)	Supplied with this task
[2]	National Instruments, “QuickSyn Microwave Frequency Synthesizer: User Guide,” Doc. No. 5580513-01, Rev. F, ECN 001779.	Supplied with this task
[3]	National Instruments, “QuickSyn Microwave Frequency Synthesizer: Calibration Software User’s Manual,” Doc. No. 5580515-01, Rev. B, ECN 001540, 2014.	Supplied with this task
[4]	National Instruments, “QuickSyn Frequency Synthesizers,” product brochure and datasheet, Oct. 2017.	Supplied with this task
[5]	National Instruments, “Clearing QuickSyn Memory,” application note.	Supplied with this task
[6]	IEEE Std 488.2-1992, *IEEE Standard Codes, Formats, Protocols, and Common Commands for Use With IEEE Std 488.1-1987, IEEE Standard Digital Interface for Programmable Instrumentation*, IEEE, New York, NY, USA, 1992.	https://ieeexplore.ieee.org/document/8705775
[7]	IEC 60488-2:2004 / IEEE 488.2, *Standard digital interface for programmable instrumentation — Part 2: Codes, formats, protocols and common commands*, IEC, Geneva, Switzerland, 2004.	https://www.vde-verlag.de/iec-normen/preview-pdf/info_iec60488-2%7Bed1.0%7Den.pdf
[8]	SCPI Consortium, *Standard Commands for Programmable Instruments (SCPI), Volume 1: Syntax and Style*, Version 1999.0, May 1999 (now maintained by the IVI Foundation).	https://www.ivifoundation.org/downloads/SCPI/scpi-99.pdf
[9]	IEEE Std 1139-2008, *IEEE Standard Definitions of Physical Quantities for Fundamental Frequency and Time Metrology — Random Instabilities*, IEEE, New York, NY, USA, 2008.	https://ieeexplore.ieee.org/document/4797525
[10]	IVI Foundation, “Specification Downloads” (SCPI and IVI specifications index).	https://www.ivifoundation.org/specifications/default.html

#	Reference	URL
[11]	Veusz Developers, “Veusz — a scientific plotting package.”	https://veusz.github.io/
[12]	Phase Matrix, Inc., “Phase Matrix Introduces a New 20 GHz QuickSyn Frequency Synthesizer,” press release, IEEE MTT-S IMS 2010, Anaheim, CA, USA, May 2010.	https://www.rfcafe.com/miscellany/press-releases/2010/phase-matrix-5-20-2010.htm
[13]	Free Software Foundation, “GNU General Public License, version 3,” 29 Jun. 2007.	https://www.gnu.org/licenses/gpl-3.0.html
[14]	SPDX Workgroup, “GNU General Public License v3.0 or later (GPL-3.0-or-later),” SPDX License List.	https://spdx.org/licenses/GPL-3.0-or-later.html
[15]	T. Preston-Werner, “Semantic Versioning 2.0.0.”	https://semver.org/spec/v2.0.0.html
[16]	Python Software Foundation, “What’s New In Python 3.13” (adds `os.process_cpu_count()`).	https://docs.python.org/3/whatsnew/3.13.html
[17]	Anaconda, Inc., conda-forge package pages for `qt.py` and `veusz`.	https://anaconda.org/conda-forge/qt.py ; https://anaconda.org/conda-forge/veusz
[18]	Veusz Developers, “Veusz 4.2.1” source distribution: `veusz/document/doc.py` (`saveToHDF5File`), `veusz/document/loader.py` (`loadHDF5Doc`), `veusz/datasets/oned.py`, `veusz/utils/textrender.py` (`latexEscape`), `veusz/veusz_main.py`, Python Package Index.	https://pypi.org/project/veusz/4.2.1/
[19]	Anaconda, Inc., conda-forge package page for `h5py`.	https://anaconda.org/conda-forge/h5py
[20]	A. Collette *et al.*, “h5py 3.16.0,” Python Package Index.	https://pypi.org/project/h5py/
[21]	W. W. Wallace, *isp_speed_monitor.py*, version 1.3.0 (`VeuszWriter`, `VszDoc`, `vz_text`, `export_veusz`, `tests/render_in_veusz.sh`), Adaptive Electrodynamics LLC dba AdaptEd, Oct. 2026.	Supplied by the author (not public)
[22]	Veusz Developers, “veusz/veusz,” GitHub repository: `master` branch (`veusz/document/commandinterpreter.py`, `veusz/veusz_listen.py`) and issue tracker, accessed 11 Oct. 2026.	https://github.com/veusz/veusz ; https://github.com/veusz/veusz/issues

#	Reference	URL
[23]	The Qt Company, “QPlainTextEdit Class,” *Qt Widgets*, Qt 6.12 documentation (`maximumBlockCount`: blocks are removed from the beginning of the document), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/qplaintextedit.html
[24]	The Qt Company, “Threads and QObjects,” Qt 6.12 documentation, section “Signals and Slots Across Threads” (auto and queued connections), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/threads-qobject.html
[25]	The Qt Company, “QTextCursor Class,” *Qt GUI*, Qt 6.12 documentation (`insertText` with a `QTextCharFormat`, `insertBlock`, `beginEditBlock`), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/qtextcursor.html
[26]	pypdf contributors, “pypdf 6.20.0,” Python Package Index, 9 Oct. 2026 (BSD-3-Clause; also on conda-forge, noarch).	https://pypi.org/project/pypdf/ ; https://anaconda.org/conda-forge/pypdf
[27]	The LaTeX Project Team, H. Oberdiek and S. Rahtz, “hyperref: Extensive support for hypertext in LaTeX,” CTAN (LPPL 1.3; 7.01h used here, 7.01s current on 11 Oct. 2026).	https://ctan.org/pkg/hyperref
[28]	TeX Users Group, “The TeX Live Guide” (LuaLaTeX engine; Windows binaries in `bin/windows`, 64-bit since TeX Live 2023), accessed 11 Oct. 2026.	https://tug.org/texlive/doc/texlive-en/texlive-en.html ; https://tug.org/texlive/windows.html
[29]	The Qt Company, “QGuiApplication Class,” Qt 6 documentation (`lastWindowClosed`: emitted when the last *visible* primary window is closed), accessed 11 Oct. 2026.	https://doc.qt.io/qt-6/qguiapplication.html
[30]	PyVISA Authors, “PyVISA 1.16.2,” Python Package Index, 27 Feb. 2026 (MIT; Python ≥ 3.10); “VISA resource names” (`ASRLboard::INSTR`, `GPIB[board]::primary address::INSTR`, `TCPIP[board]::host address::port::SOCKET`), PyVISA documentation, accessed 12 Oct. 2026.	https://pypi.org/project/PyVISA/ ; https://pyvisa.readthedocs.io/en/latest/introduction/names.html

#	Reference	URL
[31]	PyVISA Authors, “PyVISA-py 0.8.1,” Python Package Index, 4 Sep. 2025 (MIT; Python ≥ 3.10): serial instruments need PySerial, GPIB needs gpib-ctypes or linux-gpib, TCPIP needs no extra package.	https://pypi.org/project/PyVISA-py/
[32]	C. Liechti, “pySerial 3.5,” Python Package Index, 23 Nov. 2020 (BSD); documentation.	https://pypi.org/project/pyserial/ ; https://pyserial.readthedocs.io/
[33]	Python Software Foundation, “What’s New In Python 3.13,” section *time*: on Windows `monotonic()` now uses `QueryPerformanceCounter()` (1 µs) instead of `GetTickCount64()` (15.6 ms), gh-88494; `time()` likewise, gh-63207. Accessed 12 Oct. 2026.	https://docs.python.org/3/whatsnew/3.13.html
[34]	Python Software Foundation, “time — Time access and conversions” (`perf_counter`, `monotonic`, `get_clock_info`), Python 3 documentation, accessed 12 Oct. 2026.	https://docs.python.org/3/library/time.html
[35]	IBIS Open Forum, “Touchstone File Format Specification,” Version 2.1, ratified 26 Jan. 2024 (option line, `MA/DB/RI`, `[Two-Port Data Order]`, noise data), accessed 14 Oct. 2026.	https://www.ibis.org/touchstone_ver2.1/ ; https://www.ibis.org/touchstone_ver2.1/touchstone_ver2_1.pdf
[36]	D. M. Pozar, *Microwave Engineering*, 4th ed. Hoboken, NJ, USA: Wiley, 2012, Ch. 2 (lossy transmission lines) and Ch. 3 (coaxial line) (attenuation: conductor loss $\propto \sqrt{f}$ through the surface resistance, dielectric loss $\propto f$).	ISBN 978-0-470-63155-3
[37]	Phase Matrix, Inc., “A Compact Synthesizer Module Offers Instrument-grade Performance and Functionality,” *Microwave Journal*, 2 Feb. 2011 (“output power is controlled using an open-loop method”; temperature compensation), accessed 14 Oct. 2026.	https://www.microwavejournal.com/articles/10598-a-compact-synthesizer-module-offers-instrument-grade-performance-and-functionality

Sourcing confidence

Claim class	Confidence	Basis
Link parameters, terminator, 64-byte buffer	**High**	Stated verbatim in [1]
SCPI mnemonics, arguments, ranges, bit maps	**High**	[1, Table 5], reproduced byte-exact in tests
Settling delays, 5 µs dwell increment, FM interlocks	**High**	[1, Tables 2a/2b/2c]
Model coverage, power, resolution, connectors, I/O levels	**High**	[4] specification tables
Memory layout and clearing procedure	**High**	[5], explicit
+75 °C internal limit	**High**	[2], CAUTION notice
`*IDN?` field semantics	**Low**	Inferred from [1, Table 4]; the [1, Table 5] example is self-inconsistent
SCPI temperature double-issue	**Low**	Documented for native and SPI only; conservative default chosen
Response terminator	**Low**	Not documented; reader tolerates CR/LF/CRLF
Non-FSW models accepting the same SCPI set	**Medium**	[1] covers FSW-0010/0020 only; flagged in the UI
GPL-3.0 text in `LICENSE`	**High**	Byte-identical to the Debian-packaged copy of the FSF text (674 lines); gnu.org itself could not be fetched, so [13] was not compared directly
`GPL-3.0-or-later` is a current, non-deprecated SPDX identifier	**High**	SPDX machine-readable record for [14]: `isDeprecatedLicenseId: false`, OSI-approved, FSF-libre
`os.process_cpu_count` is 3.13-only	**High**	[16], and observed directly: absent on 3.12.3, present on 3.13.16
`.vszh5` layout and loader behaviour	**High**	Read from the Veusz 4.2.1 source [18] and confirmed by loading, re-saving and rendering in real Veusz 3.6.2 and 4.2.1
Veusz `Load()` and listen-mode `Quit()` limits, and the patch fixing them	**High**	Reproduced on 4.2.1; patch tested before and after; the same two functions read on `master` [22] on 2026-10-11
Windows `veusz.exe` reading a piped script in `--listen` mode	**Low**	Not tested; the in-process fallback covers it
Veusz 4.x PDF page-scale defect and its cause	**High**	Reproduced on 4.2.1 (program, in-process, Xvfb and offscreen); cause read in the 4.2.1 source and on `master` [22]; fixed by the patch and by `pdfdpi=72`, both measured
PDF bookmarks and the LuaLaTeX report	**High** on Linux, **Medium** on Windows	Built and read back here (pypdf 6.20.0, TeX Live 2023); TeX Live 2025 / MiKTeX on Windows not run; the TeX Live folder layout is from [28]

Claim class	Confidence	Basis
Third-party licence strings (§14)	**High**	Read from the installed packages' own metadata, or from the PyPI / conda-forge pages
Third-party licence *compatibility* (§14)	**Medium**	My application of the FSF's published positions; gnu.org was not fetchable, and this is not legal advice
SCPI console bounds and timings (§5.4)	**High** for this build	Measured with the same probe on 0.0.5 and 0.0.6 (PySide6 6.12, offscreen, simulated transport); absolute times depend on the PC, the ratios should not
~8 ms round-trip estimate in the rate budget	**Medium**	Engineering estimate for a 115.2 kbaud USB-CDC bridge, not vendor data; the achieved rate is measured, not estimated
Native Set Output Power format (`03` + 16-bit two's complement, ASCII hex)	**High**	[1, Table 2a] text and both worked examples, reproduced byte-exact
Power control needs option 2; Lite and mmW models have none	**High**	[4] option tables; [1] marks the native command "*When option is present"
Whether the user's unit has option 2, accepts `POW +12.0`, and whether `POW?` tracks the real output	**Unknown**	Needs the instrument and a power meter (§13 item 7)
Native Get Power (`0D`) negative-value encoding	**Medium**	Inferred from the Set Output Power examples; [1, Table 4] gives no example
PyVISA, pyvisa-py and pyserial versions, licences and back-end requirements (§2.1)	**High**	Read from their PyPI pages and documentation [30]–[32] on 12 Oct. 2026
Ethernet port 10001, GPIB address 5, USB as a standard COM port	**High**	Stated in [1], "Communication Interfaces"
IDN? field order and option-word decoding	**High** for the FSL-0020 tested	Raw reply compared with NI's soft front panels on the same unit; not yet seen on an FSW
FSL-0020 output level does not follow `POW`	**High** for that unit	Power meter, with this program and with NI's panel; the cause (no hardware, or a fixed-power calibration [3]) is **unknown**
Temperature doubling after `POW`	**Medium**	Seen four times in one session, ratio 2.00; cause unknown; not checked in NI's panel
Windows 15.6 ms clock on Python 3.12	**High**	[33], and the logged latencies take only the values 0, 15 and 16 ms
FSL-0020: `OUTP:ROSC:STAT` switches REF OUT, `STAT?` bit 5 stale	**High** for that unit	Power meter on REF OUT (user, 2026-10-14); whether `OUTP:ROSC:STAT?` answers is still unknown
NI panel steps, instruments and the ±20 Hz at 10 GHz tolerance (§3.7)	**High**	[3], read for this release

Claim class	Confidence	Basis
No published command for writing power calibration or storing to flash	**High** for [1] and [3]; **Medium** overall	Neither document gives one; NI may have it unpublished
Open-loop power control	**Medium**	Vendor article [37], not the datasheet
Cable-loss bound $\sqrt{2} \le L(20)/L(10) \le 2$	**High** for a uniform coaxial line	[36]; does not hold for a damaged connector, an adapter resonance or a sensor error
Calibration tab with real meters	**Unknown**	Tested only with fake PyVISA resources; `MEAS?` / `MEAS:FREQ?` defaults are common SCPI, not checked against a specific manual

---

13. Open items

1. **Header layout.** Author, company, email and licence are as you specified; there is deliberately no project field. The field layout and the GPL notice block are my own design. If NRAO/GBO has a mandatory header template, send it and the next revision will apply it verbatim.
2. **Copyright holder and release approval.** The files name William W. Wallace as copyright holder, as instructed. Whether NRAO/GBO or its operator holds or must approve the copyright before public release is an institutional question this package cannot settle; check it before publishing.
3. **Hardware bring-up.** On first connection to a real unit, capture `transactions.csv` from a short session and the `*IDN?` line. That settles the field order, the response terminator, the real round-trip time, and whether the SCPI temperature query needs its second issue.
4. **Environment solve and Windows render.** Create the mamba environment on the Windows machine, log a short session and use Veusz ▸ *Write .vszh5 and render*; the render is verified on Linux only. If Veusz is a standalone install, point `QUICKSYN_VEUSZ` at `veusz.exe`. To see whether the program itself works there (rather than the in-process fallback), run the export script with `--render-method program`.
5. **Veusz upstream.** Filing `upstream/VEUSZ_UPSTREAM_ISSUE.md` with the patch at [22] needs a GitHub account. Once a Veusz release carries the fixes, plain `veusz --export=out.pdf file.vszh5` will work and draw full size. The listen-mode route and `pdfdpi=72` can stay for older versions.
6. **LuaLaTeX on Windows.** Render one session with TeX Live 2025 installed and check that `quicksyn_session_report.pdf` appears beside the plots. If `lualatex` is not on the PATH, set `QUICKSYN_LUALATEX`.
7. **Output power on the real unit (0.0.7).** Done for the FSL-0020 (§3.3, §10.1): no level control. Still open for an FSW. Original procedure, not testable without the instrument. With a power meter or spectrum analyser on RF OUT: connect, note the `*IDN?` line and the reply to the native `01` (Send line); set +13, 0, −10 dBm with *Command = SCPI only*, then again with *Native only*, reading `POW?` and *Read (0D)* each time and the measured level. Then press RF OUTPUT OFF and measure. Send the `transactions.csv` of that session. This settles whether option 2 is fitted, whether the firmware accepts `POW +12.0` (sign) and `POW +0.0`, which command form works, whether `POW?` reflects the output or only the setting, and the `0D` reply format.
8. **Ethernet and GPIB transports.** Optional (§2.1): a standard-library socket transport for the Ethernet adapter (port 10001) and a PyVISA transport for GPIB (address 5) or any VISA resource. Not started; say if either is wanted.

9. **Run 0.0.10 on the FSL-0020.** Choose External with and without a 10 MHz signal at REF IN (LEDs and warnings); switch the 10 MHz output off and on with the power meter on REF OUT and note the status-bar message (SCPI confirmed, native kept, or the warning); type 0.800 GHz; try auto-read at 5 s while logging; send `transactions.csv` and `session.log`.
10. **Power on the FSL-0020.** If the level matters, run the calibration panel's *Max/Min Pwr Sweep* (§3.3) or ask NI about serial 1530200407.
11. **Temperature artefact.** In NI's main panel, note the temperature, change the power knob, and see whether its reading doubles too. If it does, it is the firmware; if not, the SCPI `DIAG:MEAS? 21` path.
12. **Qt.py.** Install it in the hardware PC's environment (`mamba install -c conda-forge qt.py`, or create the environment from `environment_quicksyn.yml`); the session ran on the direct PySide6 fallback.
13. **Calibration on the FSL-0020 (0.0.10).** With the power meter on RF OUT, run *Power vs frequency* from 0.65 to 20 GHz (typed readings are enough); first measure the cable (or its `.s2p`) so the table holds the level at RF OUT. Save, send `active_power_cal.json`. The models and VISA resources of the power meter and counter decide whether *Read power meter* and *Read counter* work as they are.
14. **Frequency floor.** Send `FREQ 650000000000`, then `FREQ?`, and measure: is 650 MHz reached, or does it clamp near 656.062 MHz like 100 MHz did? The FSL-0020's lower limit will follow.
15. **`OUTP:ROSC:STAT?`.** Send it on the console *Send* line and note the reply (or the time-out). It decides which row of the §3.5 table applies.
16. **The level shortfall (§10.2).** Before concluding that the unit is below specification: check the sensor's range and cal factor at 10 and 20 GHz, the connectors, and the path loss; the $\sqrt{f}$–f bound rules out a cable alone.

Closed in 0.0.10. *Does SCPI switch the FSL-0020's REF OUT?* Yes; only `STAT?` bit 5 is stale (§3.5). *Calibration without NI's GPIB panel:* the Calibration tab (§3.7), program-side.

Closed in 0.0.9. *Can the FSL-0020 switch REF OUT?* Yes (the user, with NI's panel and a power meter); the command path is item 9.

Closed in 0.0.8. *IDN?* field order (§9 #2). Reference-source crash. Typing below 1 GHz. Windows latency resolution. Unchecked 10 MHz output switch.

Closed in 0.0.7. *Output level could not be changed* (§3.3), as far as software can close it; the hardware part is item 7. *RF output off button* (§3.4). *Why pyserial and not PyVISA* (§2.1).

Closed in 0.0.6. *The full package:* 0.0.6 contains every file at one version; the files that existed only in the unreleased 0.0.5 build (tests, environment file, upstream patch) were rebuilt and re-verified (§10).

Closed in 0.0.5. *Licence of the code taken from `isp_speed_monitor.py`.* You own Adaptive Electrodynamics LLC dba AdaptEd, the copyright holder of `isp_speed_monitor.py`, so including the writer here under GPL-3.0-or-later is your own decision and needs no one else's consent. Both copyright lines stay in the `vszh5_writer.py` header. Licensing the same code under the GPL here does not change the terms of the speed monitor itself: a copyright holder may license its own code differently in different products. (Not legal advice.)

---

14. Licence

Copyright (C) 2026 William W. Wallace wwallace@nrao.edu, NRAO/GBO.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. It is distributed in the hope that it will be useful, but **without any warranty**; without even the implied warranty of merchantability or fitness for a particular purpose. See the full text in `LICENSE` [13].

SPDX identifier: `GPL-3.0-or-later` [14]. Each source file starts with `SPDX-License-Identifier` and `SPDX-FileCopyrightText` tags and repeats the FSF-recommended notice in its header. The notice also appears in the About dialog (Help > About) and the identity lines in `--version`.

`vszh5_writer.py` additionally carries `SPDX-FileCopyrightText: 2026 William W. Wallace, Adaptive Electrodynamics LLC dba AdaptEd`, because its design and much of its code come from `isp_speed_monitor.py` 1.3.0 [21] (see the closed item in §13).

14.1 Third-party components

Nothing below is bundled in the archive. All are installed by the environment file and imported at run time, except the TeX distribution, which is installed separately and run as a program. Licence strings are those declared in each package's own metadata (installed versions shown) or, where noted, on its PyPI or conda-forge page.

Component	Version checked	Declared licence	Role	Combinable with GPL-3.0-or-later
PySide6 / shiboken6	6.12.0	LGPL-3.0-only OR GPL-2.0-only OR GPL-3.0-only	Qt binding	Yes, via the LGPL-3.0 or GPL-3.0 option
Qt.py	2.0.7 (PyPI), 2.0.3 (conda-forge)	MIT	binding abstraction	Yes
pyserial	3.5	BSD	USB virtual COM port	Yes
NumPy	2.5.4	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	arrays, ring buffer	Yes
psutil	7.2.2	BSD-3-Clause	RAM readout	Yes
pyqtgraph	0.14.0	MIT	live strip chart	Yes
h5py	3.16.0	BSD-3-Clause	writes the `.vszh5`	Yes
pypdf	6.20.0	BSD-3-Clause	bookmarks in the rendered PDF (optional)	Yes
LuaLaTeX / TeX Live or MiKTeX	TeX Live 2023 (LuaHBTeX 1.17.0) tested	free licences, mainly LPPL 1.3 and GPL (per package)	compiles the session report as a separate program (optional; not installed by the environment file)	Yes: run as an external program, not linked
Veusz	4.2.1 (PyPI and conda-forge pages)	GPL-2.0-or-later	renders PDF/PNG from the `.vszh5` (imported, optional)	Yes, because "or later" admits version 3
CuPy	14.2.0 (PyPI page)	MIT	GPU reductions (optional)	Yes
PyVISA	1.16.2 (PyPI and conda-forge pages)	MIT	meter and counter reads, Calibration tab (optional)	Yes
PyVISA-py	0.8.1 (PyPI and conda-forge pages)	MIT	pure-Python VISA back end (optional)	Yes

The right-hand column is my application of the FSF's published compatibility positions, not legal advice; see the confidence table in §12.